

Лабораторная работа № 2

Исследование моделей SIR и Лотки–Вольтерры

Нечаева Кира Андреевна

В лабораторной работе реализованы и исследованы две системы имитационного моделирования: эпидемиологическая модель SIR и модель взаимодействия популяций Лотки–Вольтерры.

Для обеих моделей выполнено численное решение систем дифференциальных уравнений, построены графики и проведены параметрические эксперименты.

Программный код оформлен в литературном стиле. На его основе сформированы чистые Julia-скрипты, документы Quarto и Jupyter Notebook.

Содержание

1	Введение	7
2	Подготовка рабочего окружения	8
3	Модель SIR	10
3.1	Базовый эксперимент	10
3.2	Литературная версия модели SIR	12
4	Модель SIR	14
4.1	Инициализация	14
4.2	Определение модели	15
4.3	Параметры и начальные условия	16
4.4	Численное решение	16
4.5	Визуализация	20
4.6	Производительность и анализ результатов	33
4.7	Вывод	35
5	Параметрическое исследование SIR	36
6	Параметрическое исследование модели SIR	38
6.1	Модель	39
6.2	Базовые условия	39
6.3	Серия экспериментов	41
6.4	Результаты	43
7	Проверка модели SIR в Jupyter Notebook	45
8	Модель Лотки–Вольтерры	48
8.1	Базовый эксперимент	48
8.2	Литературная версия	51
9	Модель Лотки–Вольтерры	53
9.1	Инициализация	53
9.2	Определение модели	54
9.3	Параметры и начальные условия	54
9.4	Численное решение	55
9.5	Анализ и визуализация	60

9.6	Спектральный анализ	67
9.7	Анализ результатов	73
9.8	Вывод	78
10	Параметрическое исследование модели Лотки–Вольтерры	79
11	Параметрическое исследование модели Лотки–Вольтерры	82
11.1	Модель	83
11.2	Влияние α	84
11.3	Влияние γ	86
11.4	Результаты	89
12	Проверка в Jupyter Notebook	91
13	Итоговая структура проекта	93
14	Заключение	96

Список иллюстраций

Список таблиц

1 Введение

Цель лабораторной работы — освоить реализацию и исследование систем дифференциальных уравнений средствами Julia на примере моделей SIR и Лотки–Вольтерры.

В работе использовано воспроизводимое окружение DrWatson. Расчёты выполнялись с помощью пакета DifferentialEquations, а результаты сохранялись в виде данных и графиков.

Программы были дополнительно оформлены в литературном стиле, после чего из одного исходного файла автоматически формировались чистый Julia-код, документация Quarto и Jupyter Notebook.

2 Подготовка рабочего окружения

Работа выполнялась в существующем репозитории курса в ветке develop.

```
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling$ git pull origin develop
From ssh://gitverse.ru:2222/kanechaeva/2026-1-study-simulation-modeling
* branch          develop    -> FETCH_HEAD
Already up to date.
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling$ git status
On branch develop
Your branch is up to date with 'origin/develop'.

nothing to commit, working tree clean
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling$ |
```

Для лабораторной работы было создано отдельное окружение DrWatson. Перед выполнением моделей была проверена загрузка всех необходимых пакетов.

```
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project$ julia --project=. scripts/test_set_up.jl
Проверка пакетов:

✓ DrWatson
✓ DifferentialEquations
✓ SimpleDiffEq
✓ Tables
✓ DataFrames
✓ StatsPlots
✓ LaTeXStrings
✓ Plots
✓ BenchmarkTools
✓ Statistics
✓ FFTW
✓ CSV
✓ JLD2
✓ Literate
✓ IJulia
✓ Quarto

Проект: /home/kiran/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project
Каталог данных: /home/kiran/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/data
Каталог графиков: /home/kiran/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/plots
```


Структура проекта содержит отдельные каталоги для программ, данных, графиков, notebooks и документации.

```
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project$ ls scripts
intro.jl  tangle.jl  test_setup.jl
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project$ find . -maxdepth 2 -type d | sort
.
./.github
./.github/workflows
./_research
./data
./data/exp_pro
./data/exp_raw
./data/sims
./notebooks
./papers
./plots
./scripts
./src
./test
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project$
```

3 Модель SIR

3.1 Базовый эксперимент

Первая исследуемая система — трёхпараметрическая модель SIR. В ней население разделяется на восприимчивых, инфицированных и выздоровевших.

Использовались параметры:

- вероятность передачи инфекции $\beta = 0.05$;
- среднее число контактов $c = 10$;
- скорость выздоровления $\gamma = 0.25$;
- начальные условия $S_0 = 990, I_0 = 10, R_0 = 0$.

После решения системы программа автоматически определяет базовое репродуктивное число, максимальное число инфицированных, момент пика и итоговое количество переболевших.

```

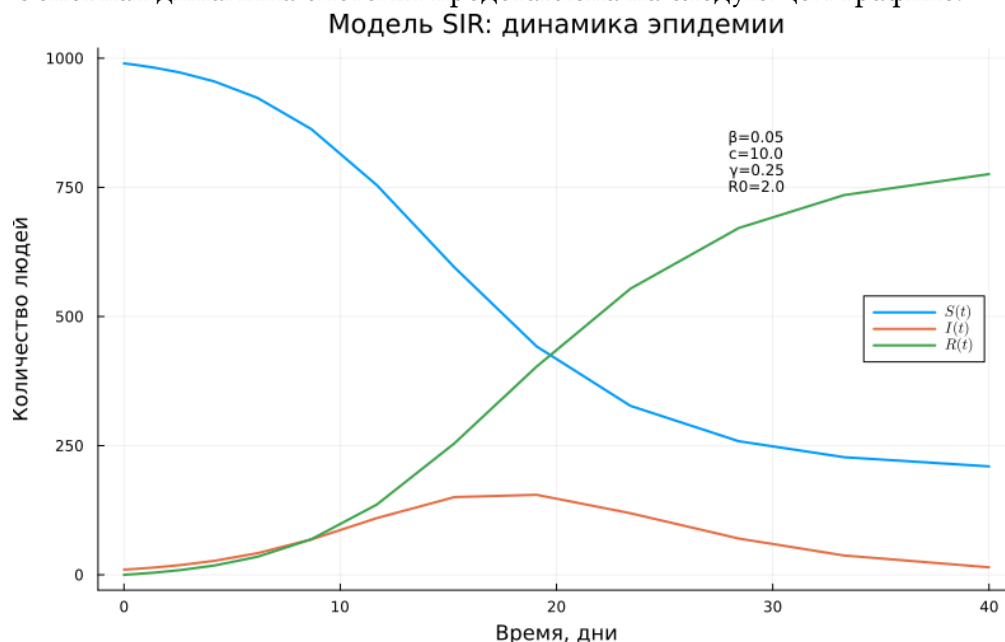
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project$ julia --project=. scripts/sir_ode.jl
Параметры модели SIR:
 $\beta = 0.05$ 
 $c = 10.0$ 
 $\gamma = 0.25$ 
 $R_0 = 2.0$ 
Средняя продолжительность болезни = 4.0 дней
Начальные условия:  $S_0 = 990.0$ ,  $I_0 = 10.0$ ,  $R_0 = 0.0$ 

=== АНАЛИЗ РЕЗУЛЬТАТОВ ===
Общая численность: 1000.0
Пиковое число заражённых: 154.8
Время пика: 19.1 дней
Итоговое число переболевших: 775.7
Медианное время решения: 0.014 мс

Моделирование SIR завершено успешно.
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project$ |

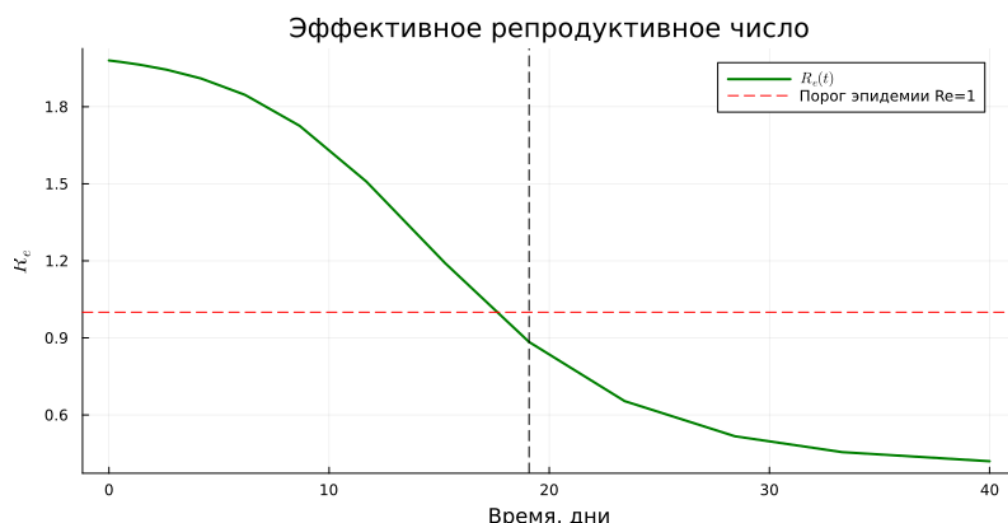
```

Основная динамика системы представлена на следующем графике.



Число восприимчивых постепенно уменьшается, число выздоровевших возрастает, а количество инфицированных сначала увеличивается, достигает максимума, после чего снижается.

Отдельно было исследовано эффективное репродуктивное число.



По мере уменьшения числа восприимчивых эффективное репродуктивное число снижается. После перехода ниже единицы дальнейший рост эпидемии прекращается.

3.2 Литературная версия модели SIR

Базовый Julia-файл был преобразован в литературный стиль. Из него автоматически сформированы исполняемый Julia-скрипт, документ Quarto и Jupyter Notebook.

```

kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project$ julia --project=. scripts/tangle.jl scripts/sir_ode.jl
Генерация из: scripts/sir_ode.jl
[ Info: generating plain script file from `~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/scripts/sir_ode.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/scripts/sir_ode/sir_ode.jl`
✓ Чистый скрипт: /home/kiran/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/scripts/sir_ode/sir_ode.jl
[ Info: generating markdown page from `~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/scripts/sir_ode.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/markdown/sir_ode/sir_ode.qmd`
✓ Quarto: /home/kiran/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/markdown/sir_ode/sir_ode.qmd
[ Info: generating notebook from `~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/scripts/sir_ode.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/notebooks/sir_ode/sir_ode.ipynb`
✓ Notebook: /home/kiran/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/notebooks/sir_ode/sir_ode.ipynb

Готово! Все файлы созданы.
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project$ |

```

Ниже интегрирована автоматически созданная Quarto-документация.

4 Модель SIR

Автор: Нечаева Кира Андреевна

Группа: НКНбд-01-23

В программе исследуется трёхпараметрическая модель SIR.

4.1 Инициализация

```
ENV["GKSwstype"] = "100"

using DrWatson
@quickactivate "project"

using DifferentialEquations
using SimpleDiffEq
using Tables
using DataFrames
using StatsPlots
using LaTeXStrings
using Plots
using BenchmarkTools
using Statistics
```

```
script_name = splitext(basename(PROGRAM_FILE))[1]
```

```
mkpath(plotsdir(script_name))
```

```
mkpath(datadir(script_name))
```

```
"/home/kiran/work/study/2026-1/2026-1==study-simulation-  
modeling/2026-1-study-simulation-modeling/labs/lab02/project/data/s
```

4.2 Определение модели

```
function sir_ode!(du, u, p, t)
```

```
    S, I, R = u
```

```
    β, c, γ = p
```

```
    N = S + I + R
```

```
    @inbounds begin
```

```
        du[1] = -β * c * I / N * S
```

```
        du[2] = β * c * I / N * S - γ * I
```

```
        du[3] = γ * I
```

```
    end
```

```
    nothing
```

```
end
```

sir_ode! (generic function with 1 method)

4.3 Параметры и начальные условия

Параметры

```
Δt = 0.1
tmax = 40.0
tspan = (0.0, tmax)

u0 = [990.0, 10.0, 0.0]

p = [
    0.05,    # Δ
    10.0,    # c
    0.25     # Δ
]
```

3-element Vector{Float64}:

```
0.05
10.0
0.25
```

Базовое репродуктивное число

```
R0 = (p[2] * p[1]) / p[3]
```

```
2.0
```

4.4 Численное решение

Решение системы


```

prob_ode = ODEProblem(
    sir_ode!,
    u0,
    tspan,
    p
)

sol_ode = solve(
    prob_ode;
    dt=Δt
)

```

retcode: Success

Interpolation: 3rd order Hermite

t: 15-element Vector{Float64}:

```

0.0
0.1
0.5331242537191838
1.3919008491040001
2.6102120755226386
4.1811820758323055
6.180123810566044
8.664951930903612
11.691425331813486
15.279370667110317
19.082963490744028
23.423425193442526
28.429493730986167
33.29877917363959

```

40.0

u: 15-element Vector{Vector{Float64}}:

```
[990.0, 10.0, 0.0]
[989.4990153200853, 10.247898072276143, 0.25308660763853114]
[987.1852964117763, 11.391111426852984, 1.4235921613706277]
[981.8312125227972, 14.026022042698543, 4.142765434504278]
[972.1392310589528, 18.75781556165419, 9.10295337939302]
[954.9905500491583, 27.00770251826761, 18.001747432574057]
[923.0706717929863, 41.92976779832723, 34.99956040868641]
[862.6705810342163, 68.49345419063427, 68.83596477514936]
[754.3022993851439, 109.74186185409295, 135.9558387607631]
[595.1014268847732, 150.41363264480702, 254.4849404704197]
[442.0141179101296, 154.8069498849187, 403.1789322049516]
[326.89511674939564, 119.06860394684863, 554.0362793037556]
[258.50852421889647, 70.11052194753057, 671.3809538335728]
[227.58338337085715, 37.328570631579815, 735.0880459975629]
[209.8412140201436, 14.473644966455762, 775.6851410134004]
```

Таблица результатов

```
df_ode = DataFrame(
    t=sol_ode.t,
    S=[u[1] for u in sol_ode.u],
    I=[u[2] for u in sol_ode.u],
    R=[u[3] for u in sol_ode.u]
)

df_ode[:, :N] =
    df_ode.S +
    df_ode.I +
```

df_ode.R

```
println("XXXXXXXXXX XXXXXX SIR:")
println("β = ", p[1])
println("c = ", p[2])
println("β = ", p[3])
println("R0 = ", round(R0, digits=3))
println(
  "XXXXXXXXXX XXXXXXXXXXXXXXXXXXXX XXXXXXXX = ",
  round(1 / p[3], digits=2),
  " XXXXX"
)

println(
  "XXXXXXXXXX XXXXXXXX: S0 = ",
  u0[1],
  ", I0 = ",
  u0[2],
  ", R0 = ",
  u0[3]
)
```

XXXXXXXXXX XXXXXX SIR:

$\beta = 0.05$

$c = 10.0$

$\beta = 0.25$

$R0 = 2.0$

XXXXXXXXXX XXXXXXXXXXXXXXXXXXXX XXXXXXXX = 4.0 XXXXX

XXXXXXXXXX XXXXXXXX: $S0 = 990.0$, $I0 = 10.0$, $R0 = 0.0$

4.5 Визуализация

Основной график

```
sir_matrix =  
    hcat(  
        df_ode.S,  
        df_ode.I,  
        df_ode.R  
    )  
  
plt1 = plot(  
    df_ode.t,  
    sir_matrix,  
    label=[L"S(t)" L"I(t)" L"R(t)"],  
    xlabel="Время, лет",  
    ylabel="Численность населения",  
    title="Модель SIR: Численность населения",  
    linewidth=2,  
    legend=:right,  
    grid=true,  
    size=(800, 500)  
)  
  
annotate!(  
    plt1,  
    maximum(df_ode.t) * 0.7,  
    maximum(df_ode.N) * 0.8,  
    text(
```

```

"n=${p[1]}\nc=${p[2]}\nn=${p[3]}\nR0=$(round(R0,digits=2))",
    8,
    :left
)
)

```

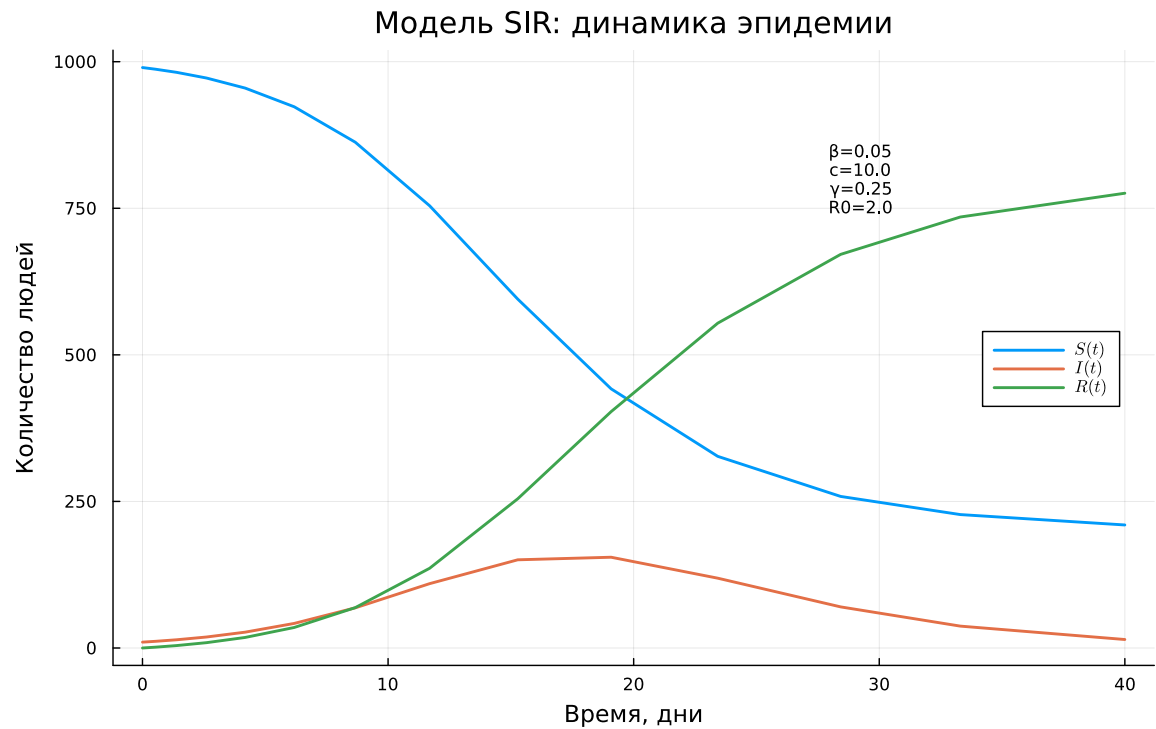


График заражённых

```

plt2 = plot(
    df_ode.t,
    df_ode.I,
    label=L"I(t)",
    xlabel="Время, дни",
    ylabel="Количество заражённых",
    title="Модель SIR: динамика эпидемии",
    color=:red,
    linewidth=2,

```

```

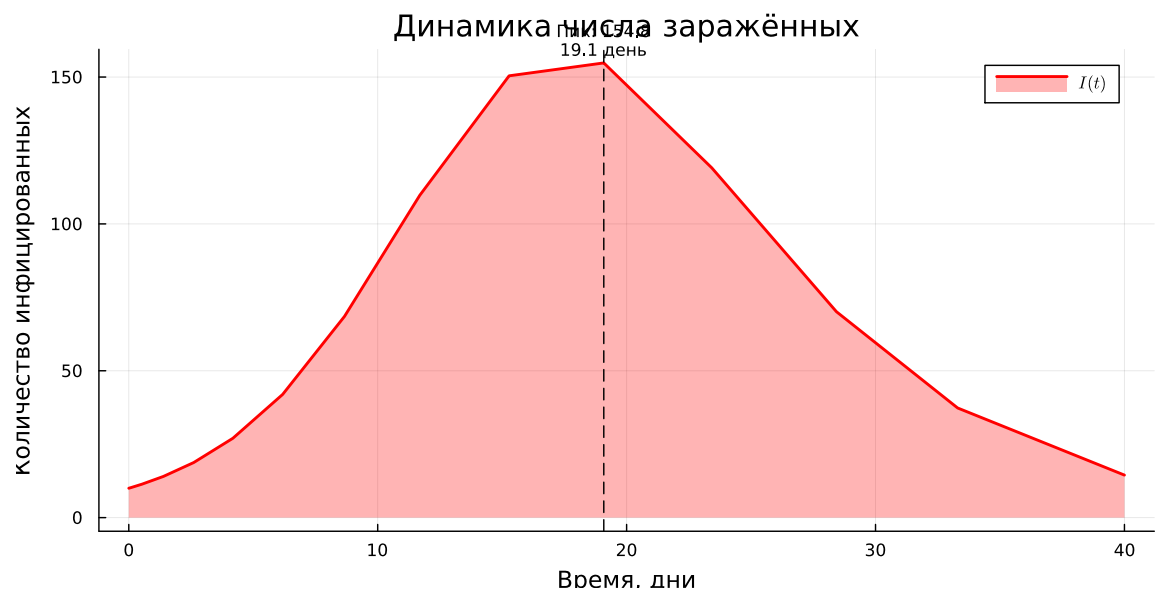
        fill=(0, 0.3, :red),
        grid=true,
        size=(800, 400)
    )

    peak_idx = argmax(df_ode.I)
    peak_time = df_ode.t[peak_idx]
    peak_value = df_ode.I[peak_idx]

    vline!(
        plt2,
        [peak_time],
        color=:black,
        linestyle=:dash,
        label=false
    )

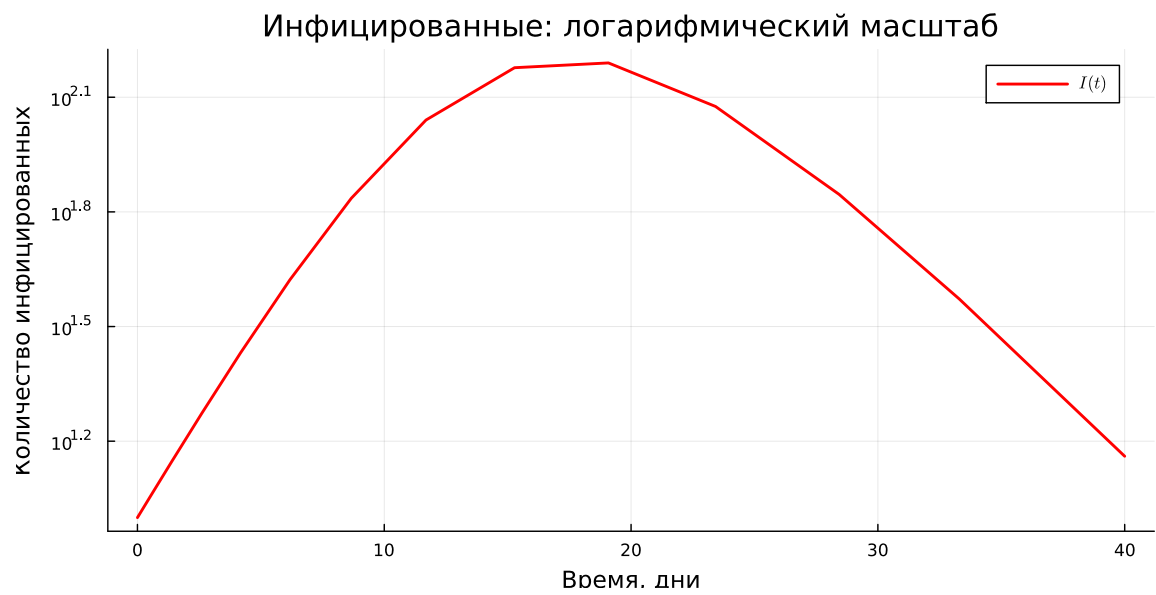
    annotate!(
        plt2,
        peak_time,
        peak_value * 1.05,
        text(
            "xxx: $(round(peak_value,digits=1))\n$(round(peak_time,digits=1)
            8
        )
    )
)

```



Логарифмический масштаб

```
plt3 = plot(
    df_ode.t,
    df_ode.I,
    label=L" $I(t)$ ",
    xlabel="Время, дни",
    ylabel="количество инфицированных",
    title="Динамика числа заражённых",
    yscale=:log10,
    color=:red,
    linewidth=2,
    grid=true,
    size=(800, 400)
)
```



Доли популяции

```
percentages =
    hcat(
        df_ode.S,
        df_ode.I,
        df_ode.R
    ) ./ df_ode.N .* 100

plt4 = plot(
    df_ode.t,
    percentages,
    label=[
        L"S(t)/N"
        L"I(t)/N"
        L"R(t)/N"
    ],
    xlabel="XXXXX, XXX",
```



```

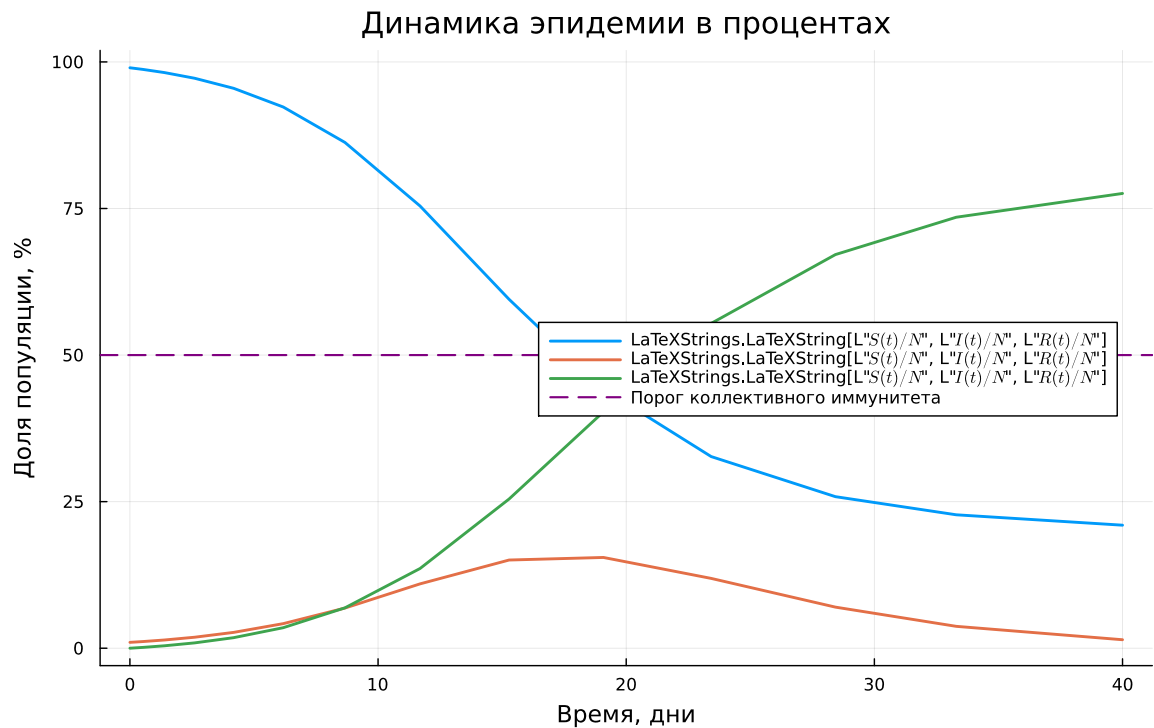
ylabel="Susceptible, %",
title="Susceptible, Infected, and Recovered",
linewidth=2,
legend=:right,
grid=true,
size=(800, 500)
)

if R0 > 1

    herd_immunity_threshold =
        (1 - 1/R0) * 100

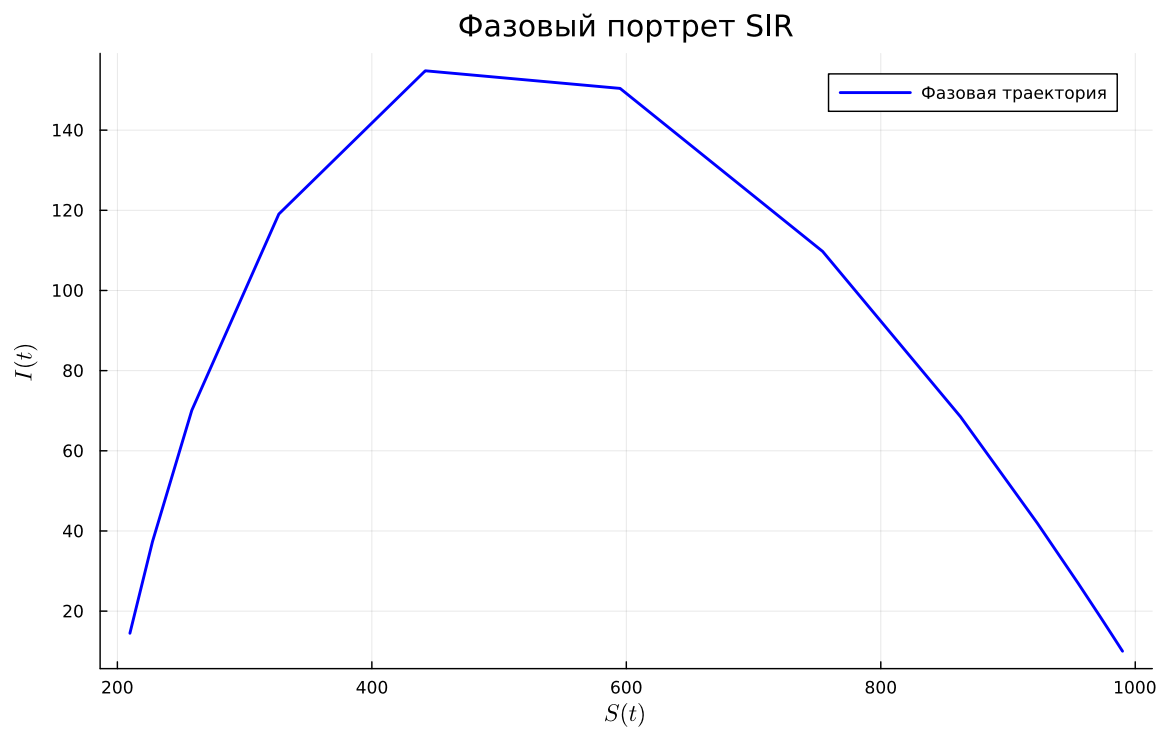
    hline!(
        plt4,
        [herd_immunity_threshold],
        color=:purple,
        linestyle=:dash,
        label="Herd Immunity Threshold",
        linewidth=1.5
    )
end

```



Фазовый портрет

```
plt5 = plot(
    df_ode.S,
    df_ode.I,
    label="S(t) I(t)",
    xlabel=L"S(t)",
    ylabel=L"I(t)",
    title="SIR",
    color=:blue,
    linewidth=2,
    grid=true,
    size=(800, 500)
)
```



Эффективное репродуктивное число

```
df_ode[:, :Re] =
    R0 .* df_ode.S ./ df_ode.N

plt6 = plot(
    df_ode.t,
    df_ode.Re,
    label=L"R_e(t)",
    xlabel="Время, год",
    ylabel=L"R_e",
    title="Эффективное репродуктивное число",
    color=:green,
    linewidth=2,
    grid=true,
    size=(800, 400))
```

```

)

hline!(
  plt6,
  [1.0],
  color=:red,
  linestyle=:dash,
  label="XXXXXXXX Re=1"
)

cross_idx =
  findfirst(
    x -> x < 1,
    df_ode.Re
  )

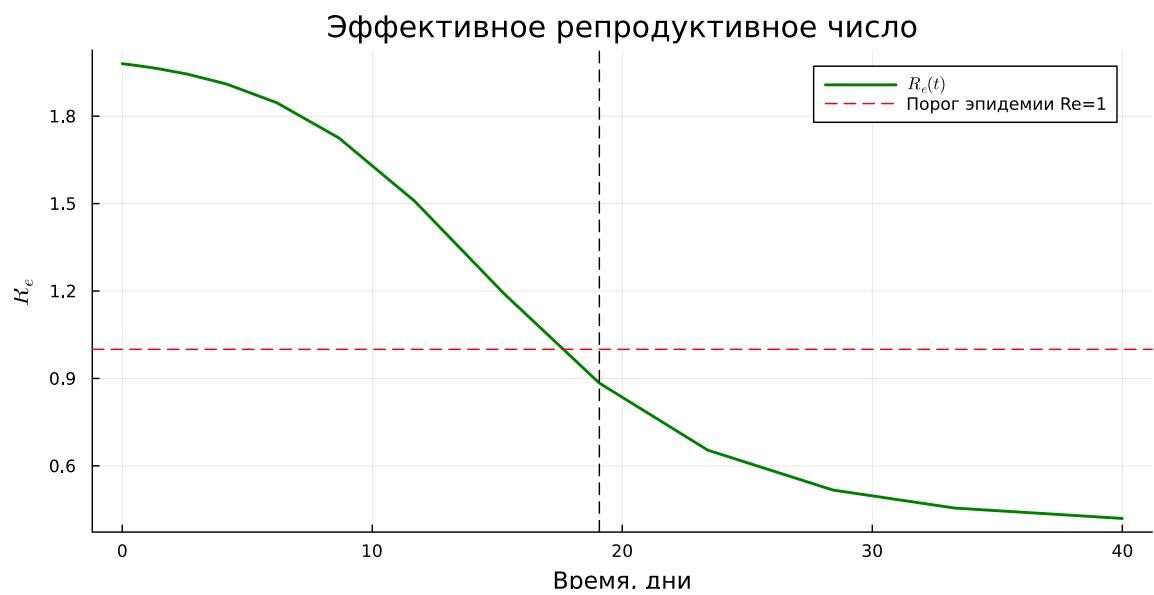
if !isnothing(cross_idx) && cross_idx > 1

  cross_time =
    df_ode.t[cross_idx]

  vline!(
    plt6,
    [cross_time],
    color=:black,
    linestyle=:dash,
    label=false
  )

```

end



Общая панель

```
plt7 = plot(  
    layout=(2, 3),  
    size=(1200, 800)  
)  
  
plot!(  
    plt7[1],  
    df_ode.t,  
    df_ode.S,  
    label=L"S(t)",  
    title="XXXXXXXXXXXXXXXXXX"  
)  
  
plot!(  
    plt7[2],
```

```

df_ode.t,
df_ode.I,
label=L"I(t)",
title="XXXXXXXXXXXXXXXXX"
)

plot!(
    plt7[3],
    df_ode.t,
    df_ode.R,
    label=L"R(t)",
    title="XXXXXXXXXXXXXXXXX"
)

plot!(
    plt7[4],
    df_ode.t,
    df_ode.I,
    yscale=:log10,
    label=L"I(t)",
    title="XXX. XXXXXXXXX"
)

plot!(
    plt7[5],
    df_ode.S,
    df_ode.I,
    label=false,

```

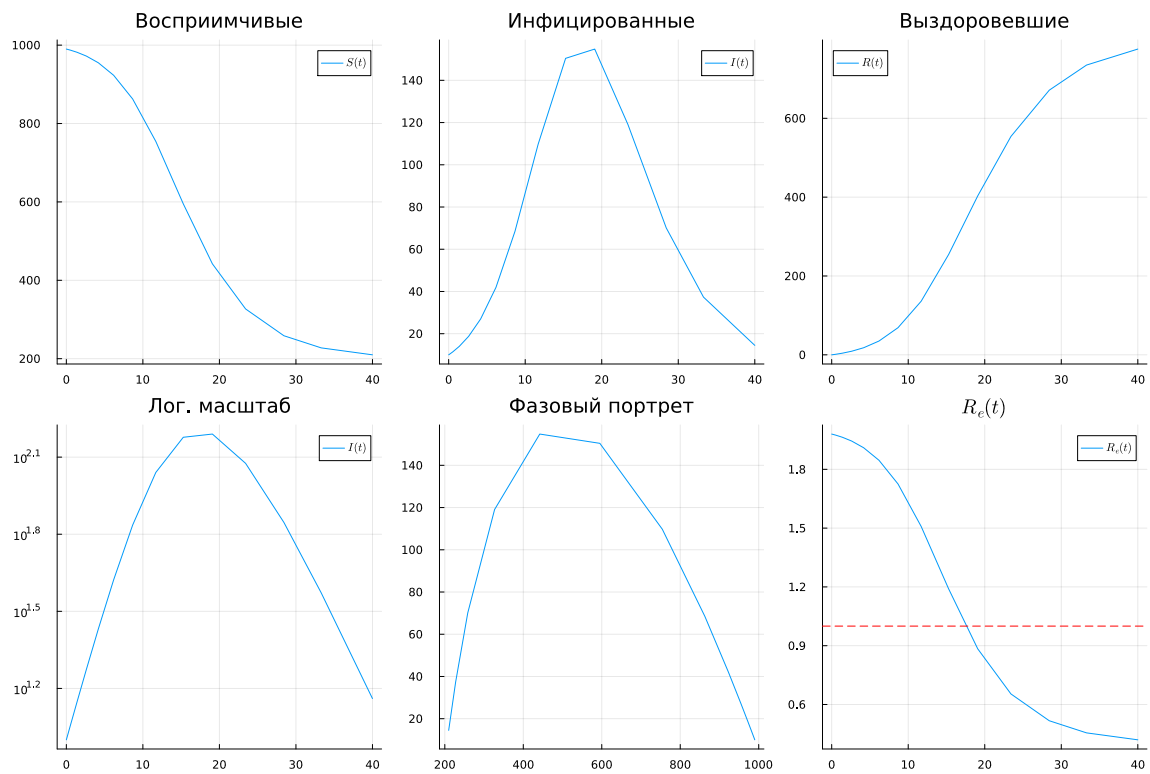
```

        title="XXXXXXXX XXXXXXXX"
    )

plot!(
    plt7[6],
    df_ode.t,
    df_ode.Re,
    label=L"R_e(t)",
    title=L"R_e(t)"
)

hline!(
    plt7[6],
    [1.0],
    linestyle=:dash,
    color=:red,
    label=false
)

```



Сохранение

```
savefig(
    plt1,
    plotsdir(script_name, "sir_main.png")
)

savefig(
    plt2,
    plotsdir(script_name, "sir_infected.png")
)

savefig(
    plt3,
    plotsdir(script_name, "sir_log_scale.png")
)
```



```

)

savefig(
    plt4,
    plotsdir(script_name, "sir_percentages.png")
)

savefig(
    plt5,
    plotsdir(script_name, "sir_phase_portrait.png")
)

savefig(
    plt6,
    plotsdir(script_name, "sir_effective_R.png")
)

savefig(
    plt7,
    plotsdir(script_name, "sir_panel.png")
)

```

"/home/kiran/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/plots/

4.6 Производительность и анализ результатов

Бенчмарк


```

    "  ")

println("\nSIR  ")

```

```

===  ===
: 1000.0
: 154.8
: 19.1
: 775.7
: 0.013

SIR .

```

4.7 Вывод

Модель позволяет определить динамику эпидемии, положение пика и момент снижения эффективного репродуктивного числа ниже единицы.

5 Параметрическое исследование SIR

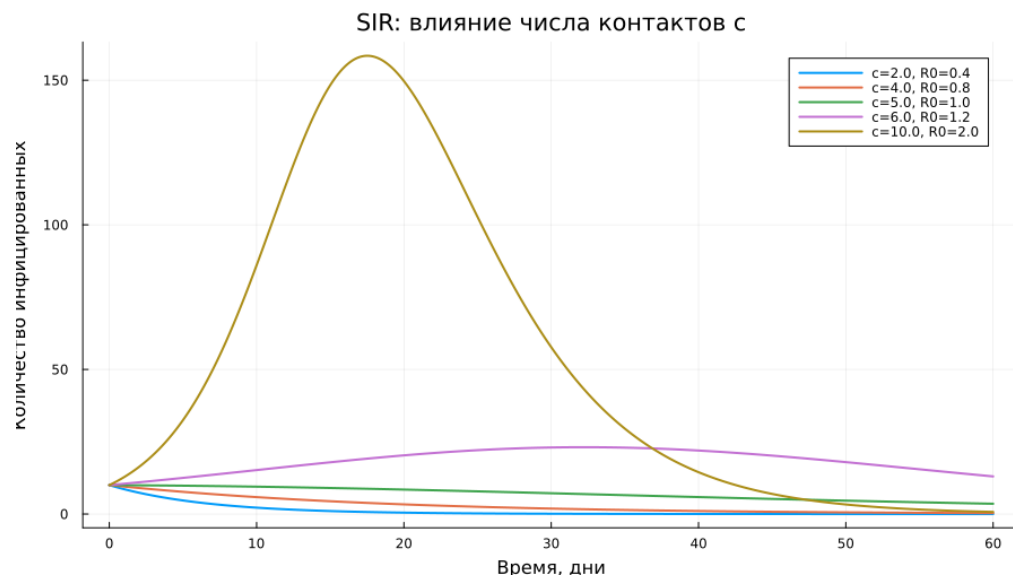
Для изучения влияния интенсивности социальных контактов было выполнено несколько расчётов при различных значениях параметра c . Остальные условия модели сохранялись неизменными.

```
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project$ julia --project=. scripts/sir_ode_params.jl
Параметрическое исследование SIR:

5x5 DataFrame
  Row  | c      R0      peak_I      peak_time      final_R
      | Float64 Float64 Float64      Float64      Float64
-----+-----
  1    | 2.0    0.4    10.0          0.0    16.5187
  2    | 4.0    0.8    10.0          0.0    43.641
  3    | 5.0    1.0    10.0          0.0   106.117
  4    | 6.0    1.2   23.1073    32.2   276.743
  5    | 10.0   2.0  158.452     17.5  798.964

Параметрическое исследование SIR завершено.
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project$
```

Сравнение траекторий инфицированных:



Изменение числа контактов непосредственно изменяет базовое репродуктивное число и характер эпидемической динамики.

Параметрическая версия также была преобразована в Julia, Quarto и Jupyter Notebook.

6 Параметрическое исследование модели SIR

Автор: Нечаева Кира Андреевна

Исследуем влияние среднего числа контактов C .

```
ENV["GKSwstype"] = "100"

using DrWatson
@quickactivate "project"

using DifferentialEquations
using DataFrames
using Plots
using CSV
using JLD2

script_name =
    splitext(basename(PROGRAM_FILE))[1]

mkpath(plotsdir(script_name))
mkpath(datadir(script_name))

"/home/kiran/work/study/2026-1/2026-1==study-simulation-
```

6.1 Модель

```
function sir_ode!(du, u, p, t)

    S, I, R = u
    β, c, γ = p

    N = S + I + R

    du[1] = -β * c * I / N * S
    du[2] = β * c * I / N * S - γ * I
    du[3] = γ * I

    nothing
end
```

sir_ode! (generic function with 1 method)

6.2 Базовые условия

```
u0 = [990.0, 10.0, 0.0]
tspan = (0.0, 60.0)

β = 0.05
γ = 0.25
```

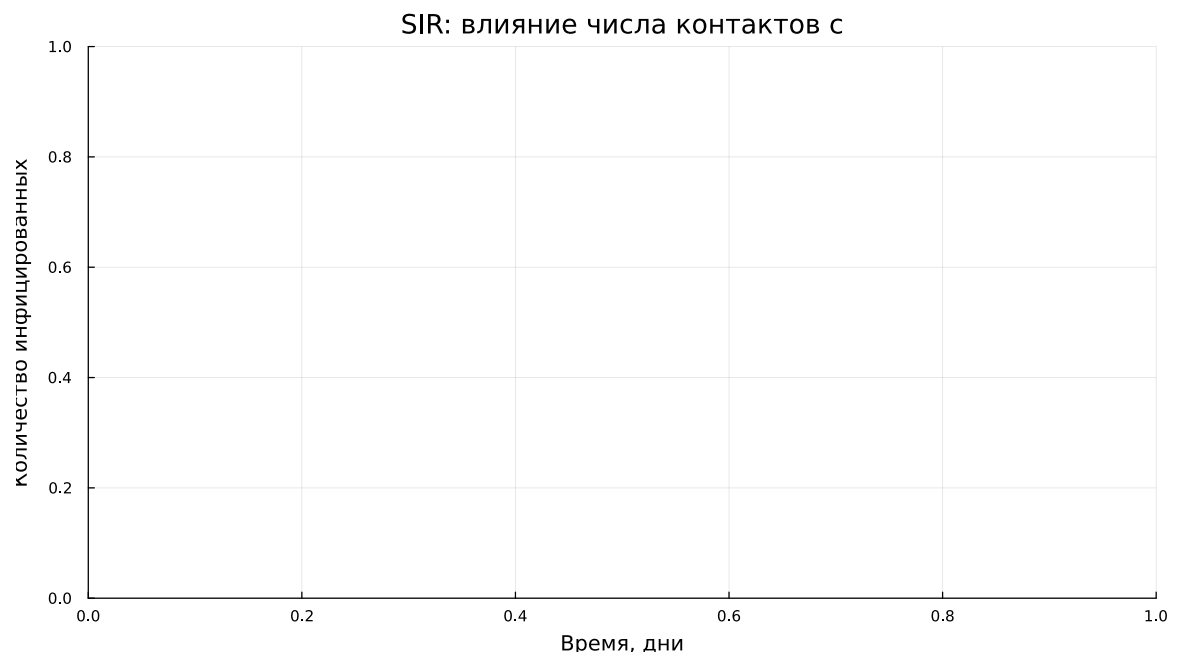
```

c_values =
    [2.0, 4.0, 5.0, 6.0, 10.0]

results = DataFrame(
    c=Float64[],
    R0=Float64[],
    peak_I=Float64[],
    peak_time=Float64[],
    final_R=Float64[]
)

plt = plot(
    xlabel="Time, t",
    ylabel="SIR: S, I, R",
    title="SIR: S, I, R vs c",
    linewidth=2,
    grid=true,
    size=(900, 500)
)

```

6.3 Серия экспериментов

```
for c in c_values

    p = [0, c, 0]

    prob =
        ODEProblem(
            sir_ode!,
            u0,
            tspan,
            p
        )

    sol =
```

```

    solve(
        prob,
        Tsit5();
        saveat=0.1
    )

    I =
        [u[2] for u in sol.u]

    R =
        [u[3] for u in sol.u]

    idx = argmax(I)

    R0_value =
        c * 1 / 1

    push!(
        results,
        (
            c,
            R0_value,
            I[idx],
            sol.t[idx],
            R[end]
        )
    )

```

```
plot!(  
    plt,  
    sol.t,  
    I,  
    label="c=$c, R0=$(round(R0_value,digits=2))",  
    linewidth=2  
)  
end
```

6.4 Результаты

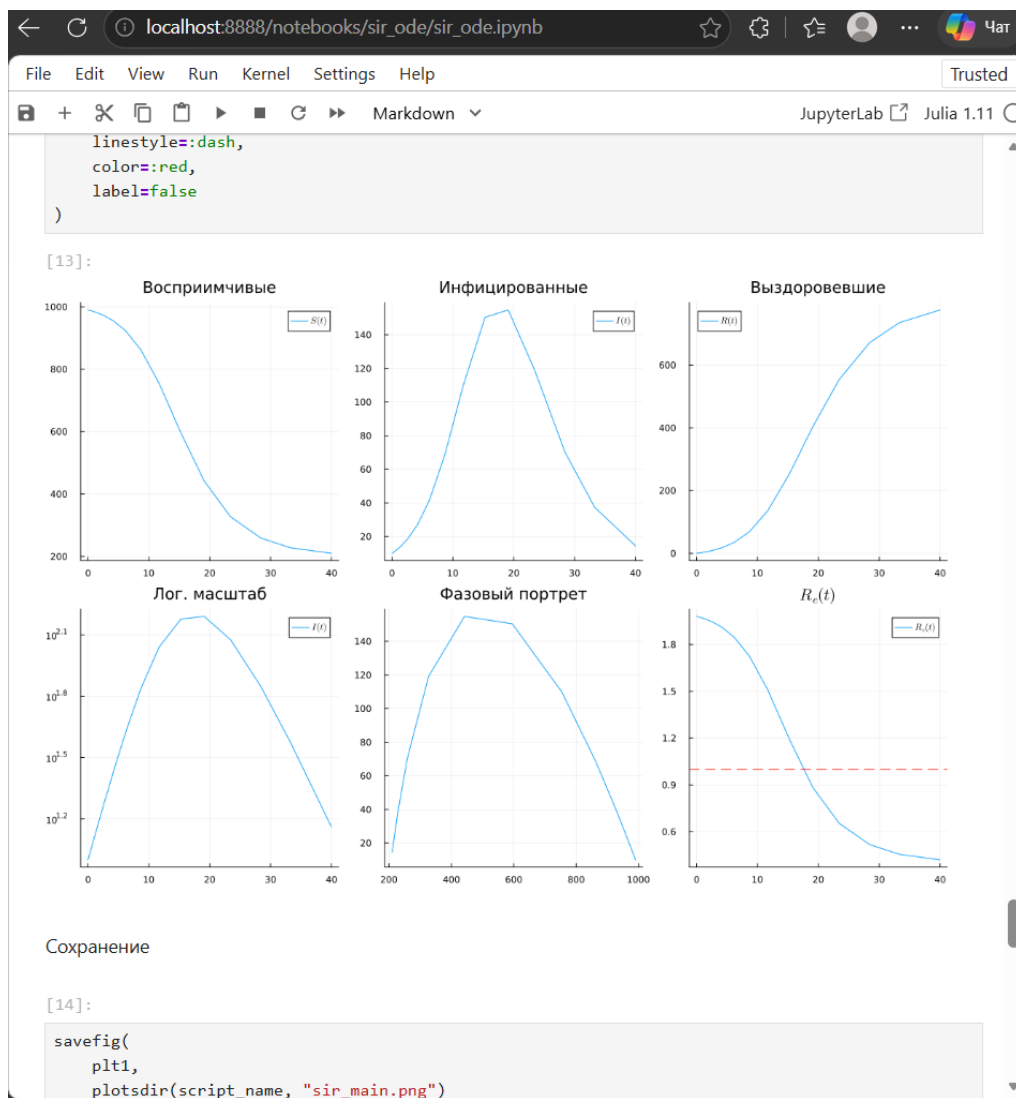
```
println("XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX SIR:\n")
println(results)

savefig(
    plt,
    plotsdir(
        script_name,
        "sir_contact_scan.png"
    )
)

CSV.write(
    datadir(
        script_name,
        "sir_contact_scan.csv"
    ),
    results
```


7 Проверка модели SIR в Jupyter Notebook

Сгенерированный notebook базовой версии был выполнен с использованием ядра Julia.



Таким же способом была выполнена параметрическая версия модели.

localhost:8888/notebooks/sir_ode_params/sir_ode_params.ipynb

File Edit View Run Kernel Settings Help Trusted

JupyterLab Julia 1.11

```

plotsdir(
    script_name,
    "sir_contact_scan.png"
)
)

CSV.write(
    datadir(
        script_name,
        "sir_contact_scan.csv"
    ),
    results
)

@save datadir(
    script_name,
    "sir_contact_scan.jld2"
) results

println(
    "\nПараметрическое исследование SIR завершено."
)

```

Параметрическое исследование SIR:

5x5 DataFrame

Row	c	R0	peak_I	peak_time	final_R
	Float64	Float64	Float64	Float64	Float64
1	2.0	0.4	10.0	0.0	16.5187
2	4.0	0.8	10.0	0.0	43.641
3	5.0	1.0	10.0	0.0	106.117
4	6.0	1.2	23.1073	32.2	276.743
5	10.0	2.0	158.452	17.5	798.964

Параметрическое исследование SIR завершено.

8 Модель Лотки–Вольтерры

8.1 Базовый эксперимент

Вторая модель описывает взаимодействие двух популяций: жертв и хищников.

В работе использовались параметры

- $\alpha = 0.1$ — естественный прирост жертв;
- $\beta = 0.02$ — интенсивность взаимодействия с хищниками;
- $\delta = 0.01$ — эффективность роста популяции хищников;
- $\gamma = 0.3$ — естественная смертность хищников.

Начальные значения популяций составляли 40 жертв и 9 хищников.

Результаты запуска программы представлены ниже.


```

(jupyter) kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2
026-1-study-simulation-modeling/labs/lab02/project$ julia --project=. script
s/lv_ode.jl
[ Warning: Verbosity toggle: dense_output_saveat
  Dense output is incompatible with saveat. Please use the SavingCallback f
rom the Callback Library to mix the two behaviors.
L @ OrdinaryDiffEqCore ~/.julia/packages/OrdinaryDiffEqCore/src/solve.
jl:410
=====
Модель Лотки-Вольтерры
=====

Параметры:
 $\alpha = 0.1$ 
 $\beta = 0.02$ 
 $\delta = 0.01$ 
 $\gamma = 0.3$ 

Начальные условия:
Жертвы = 40.0
Хищники = 9.0

Стационарная точка:
 $x^* = 30.0$ 
 $y^* = 5.0$ 

Доминирующая частота жертв: 0.025
Период колебаний жертв: 40.02

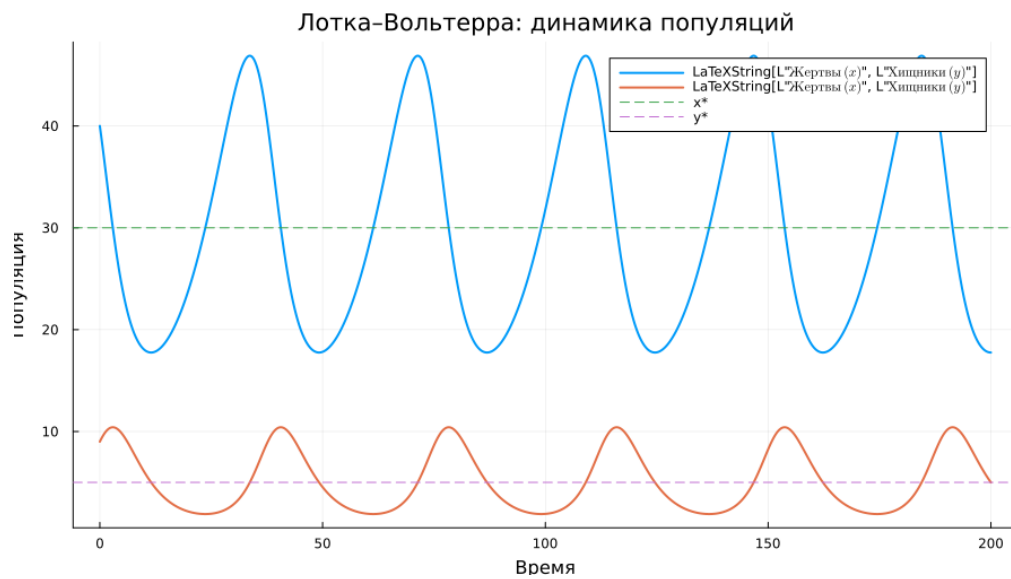
=====
Анализ результатов
=====
Жертвы: min=17.75, max=46.89, mean=29.71
Хищники: min=1.9, max=10.41, mean=5.2

Первый пик жертв: t=33.7, значение=46.89
Первый пик хищников: t=2.9, значение=10.41
Сдвиг фаз: -30.8

Моделирование завершено успешно!
(jupyter) kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2
026-1-study-simulation-modeling/labs/lab02/project$ |

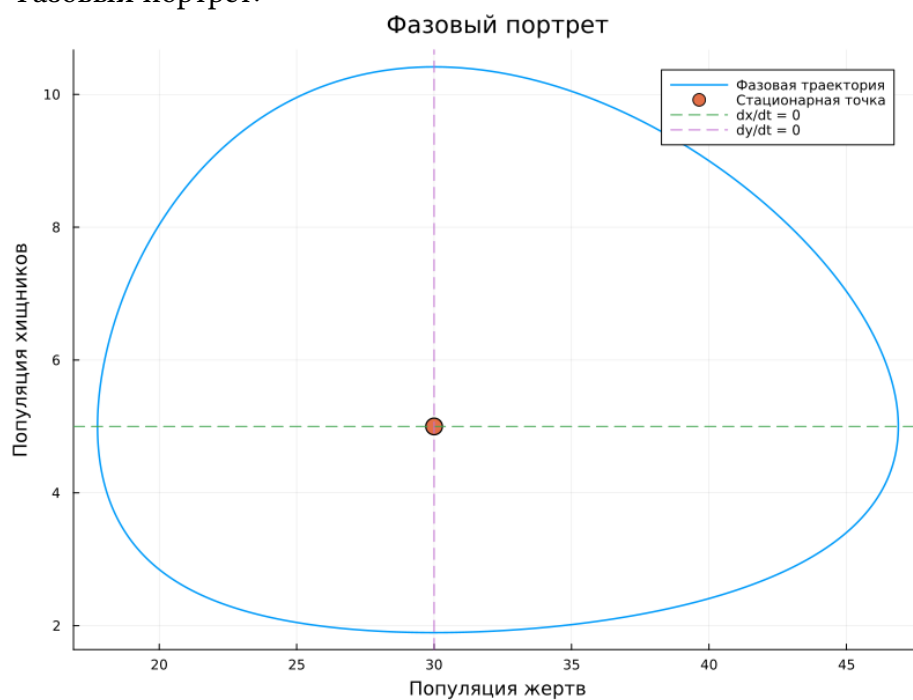
```

Динамика обеих популяций:



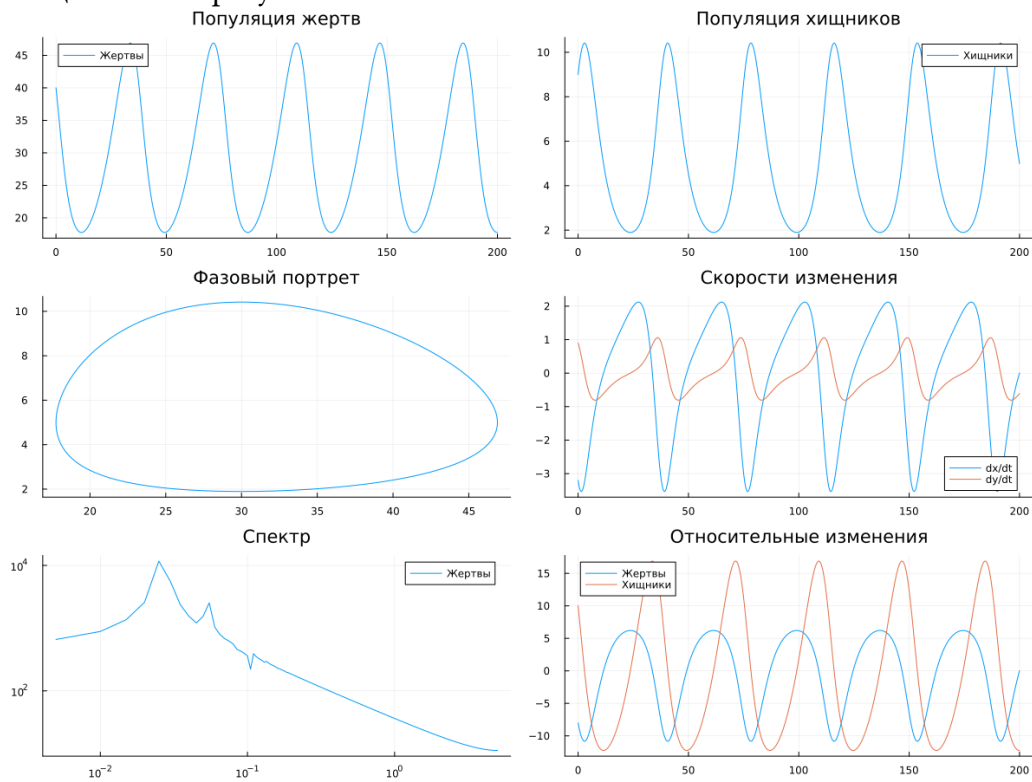
Рост численности жертв создаёт условия для последующего роста численности хищников, поэтому между максимумами двух популяций возникает временной сдвиг.

Фазовый портрет:



Замкнутая траектория на фазовой плоскости отражает циклический характер взаимодействия двух популяций.

Общая панель результатов:



8.2 Литературная версия

Исходная программа также была преобразована в литературный стиль.

```

kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project$ julia --project=. scripts/tangle.jl scripts/lv_ode.jl
Генерация из: scripts/lv_ode.jl
[ Info: generating plain script file from `~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/scripts/lv_ode.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/scripts/lv_ode/lv_ode.jl`
      ✓ Чистый скрипт: /home/kiran/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/scripts/lv_ode/lv_ode.jl
[ Info: generating markdown page from `~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/scripts/lv_ode.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/markdown/lv_ode/lv_ode.qmd`
      ✓ Quarto: /home/kiran/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/markdown/lv_ode/lv_ode.qmd
[ Info: generating notebook from `~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/scripts/lv_ode.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/notebooks/lv_ode/lv_ode.ipynb`
      ✓ Notebook: /home/kiran/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project/notebooks/lv_ode/lv_ode.ipynb

Готово! Все файлы созданы.
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project$ |

```

Автоматически сформированная документация:

9 Модель Лотки–Вольтерры

Автор: Нечаева Кира Андреевна

Группа: НКНбд-01-23

Исследуется классическая система взаимодействия хищников и жертв.

9.1 Инициализация

```
ENV["GKSwstype"] = "100"

using DrWatson
@quickactivate "project"

using DifferentialEquations
using DataFrames
using StatsPlots
using LaTeXStrings
using Plots
using Statistics
using FFTW

script_name =
    splitext(basename(PROGRAM_FILE))[1]
```

```
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

```
"/home/kiran/work/study/2026-1/2026-1==study-simulation-
modeling/2026-1-study-simulation-modeling/labs/lab02/project/data/s
```

9.2 Определение модели

```
function lotka_volterra!(du, u, p, t)

    x, y = u
     $\alpha$ ,  $\beta$ ,  $\gamma$ ,  $\delta$  = p

    @inbounds begin
        du[1] =  $\alpha$ *x -  $\beta$ *x*y
        du[2] =  $\beta$ *x*y -  $\delta$ *y
    end

    nothing
end
```

lotka_volterra! (generic function with 1 method)

9.3 Параметры и начальные условия

Параметры

```

p_lv = [
    0.1,    # ⓧ
    0.02,   # ⓧ
    0.01,   # ⓧ
    0.3     # ⓧ
]

u0_lv = [40.0, 9.0]

tspan_lv = (0.0, 200.0)

dt_lv = 0.01
saveat_lv = 0.1

```

0.1

9.4 Численное решение

Задача

```

prob_lv =
    ODEProblem(
        lotka_volterra!,
        u0_lv,
        tspan_lv,
        p_lv
    )

sol_lv =

```

```

solve(
    prob_liv,
    Tsit5();
    dt=dt_liv,
    reltol=1e-8,
    abstol=1e-10,
    saveat=saveat_liv,
    dense=true
)

```

└ Warning: Verbosity toggle: dense_output_saveat

⊠ Dense output is incompatible with saveat. Please use the SavingCallback

└ @ OrdinaryDiffEqCore ~/.julia/packages/OrdinaryDiffEqCore/niPe0/src

retcode: Success

Interpolation: specialized 4th order "free" interpolation

t: 2001-element Vector{Float64}:

0.0

0.1

0.2

0.3

0.4

0.5

0.6

0.7

0.8

0.9

⊠

199.2

199.3

199.4

199.5

199.6

199.7

199.8

199.9

200.0

u: 2001-element Vector{Vector{Float64}}:

[40.0, 9.0]

[39.67773201890384, 9.088990237207637]

[39.35113789503168, 9.175882726306998]

[39.02053587808482, 9.260562174341327]

[38.68624820281499, 9.342916311942014]

[38.34860018759227, 9.422836265415745]

[38.007919320045765, 9.5002169187062]

[37.664534369328166, 9.574957250330058]

[37.31877446654823, 9.646960666854817]

[36.970968217678404, 9.716135308317774]

⊠

[17.82435314492932, 5.5080193185067206]

[17.807442037033763, 5.44131515455156]

[17.79290768632072, 5.37533430004016]

[17.780718473179295, 5.310082577777476]

[17.770843588540156, 5.245565084811861]

[17.763253013148624, 5.1817862454808905]

[17.7579175174593, 5.118749811720926]

[17.75480865796657, 5.056458873592422]

[17.753898747959717, 4.994915937053436]

Таблица данных

```
df_lv = DataFrame()

df_lv[!, :t] =
    sol_lv.t

df_lv[!, :prey] =
    [u[1] for u in sol_lv.u]

df_lv[!, :predator] =
    [u[2] for u in sol_lv.u]

df_lv[!, :dprey_dt] =
    p_lv[1] .* df_lv.prey .-
    p_lv[2] .* df_lv.prey .* df_lv.predator

df_lv[!, :dpredator_dt] =
    p_lv[3] .* df_lv.prey .* df_lv.predator .-
    p_lv[4] .* df_lv.predator

println("="^60)
println("XXXXXXXX XXXX-XXXXXXXX")
println("="^60)

println("\nXXXXXXXX:")
println("X = ", p_lv[1])
println("X = ", p_lv[2])
println("X = ", p_lv[3])
```

```
println("ϕ = ", p_lv[4])

println("\nXXXXXXXXXX XXXXXXXX:")
println("XXXXXX = ", u0_lv[1])
println("XXXXXX = ", u0_lv[2])
```

```
=====
XXXXXXXX XXXXX-XXXXXXXX
=====
```

```
XXXXXXXXXX:
ϕ = 0.1
ϕ = 0.02
ϕ = 0.01
ϕ = 0.3
```

```
XXXXXXXXXX XXXXXXXX:
XXXXXX = 40.0
XXXXXX = 9.0
```

Стационарная точка

```
x_star =
    p_lv[4] / p_lv[3]

y_star =
    p_lv[1] / p_lv[2]

println("\nXXXXXXXXXXXXXXXX XXXXX:")
```

```
println("x* = ", round(x_star, digits=3))
println("y* = ", round(y_star, digits=3))
```

XXXXXXXXXXXX XXXX:

$x^* = 30.0$

$y^* = 5.0$

9.5 Анализ и визуализация

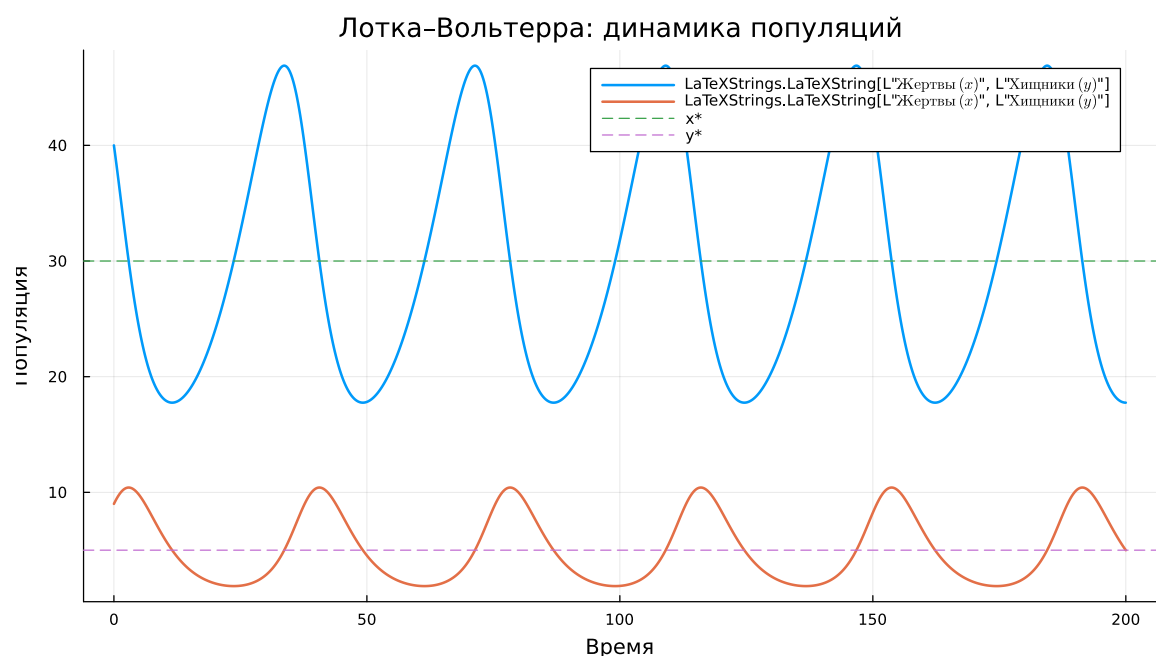
1. Динамика

```
plt1 = plot(
    df_lv.t,
    hcat(
        df_lv.prey,
        df_lv.predator
    ),
    label=[
        L"XXXXX\ (x)"
        L"XXXXXX\ (y)"
    ],
    xlabel="XXXX",
    ylabel="XXXXXXXX",
    title="XXXX-XXXXXX: XXXXX XXXXXXX",
    linewidth=2,
    legend=:topright,
    grid=true,
    size=(900, 500)
```

```
)

hline!(
    plt1,
    [x_star],
    linestyle=:dash,
    label="x* "
)

hline!(
    plt1,
    [y_star],
    linestyle=:dash,
    label="y* "
)
)
```



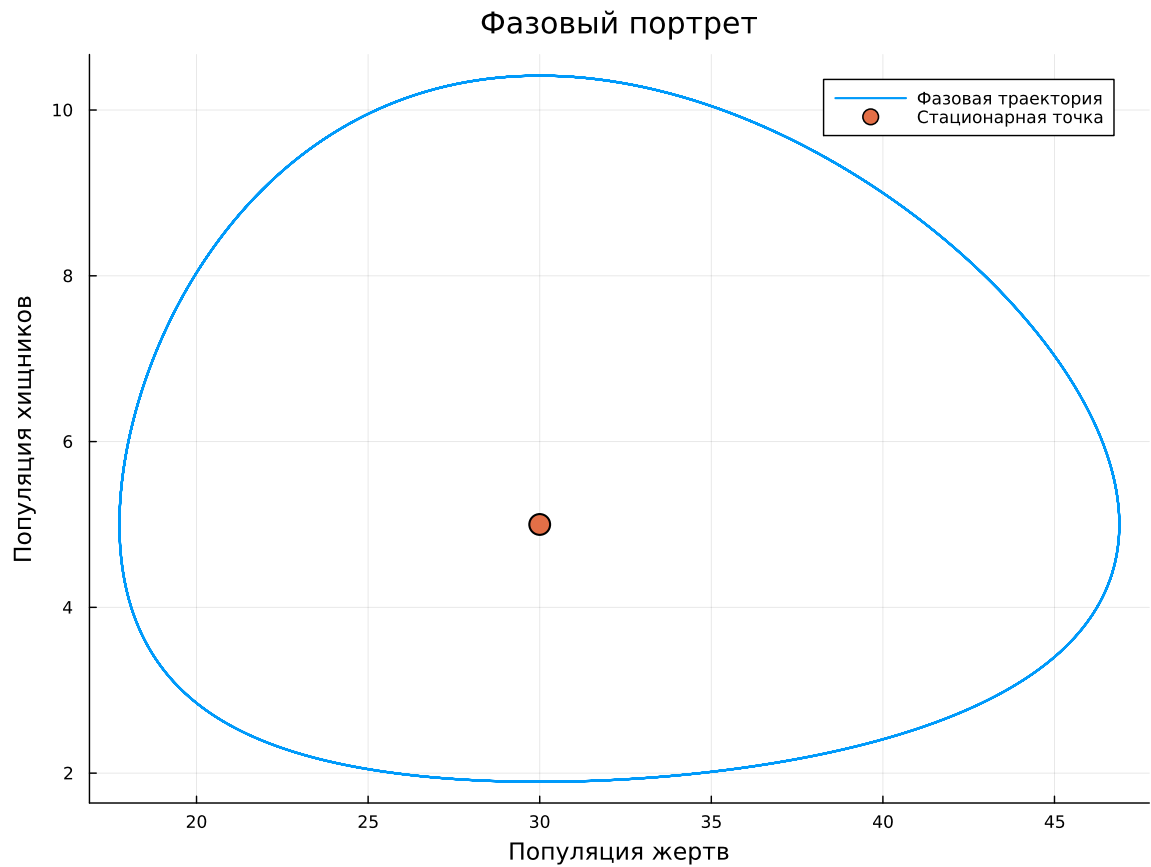
2. Фазовый портрет

```

plt2 = plot(
    df_lv.prey,
    df_lv.predator,
    label="XXXXXXXX XXXXXXXXXXXX",
    xlabel="XXXXXXXXXX XXXX",
    ylabel="XXXXXXXXXX XXXXXXXXXXXX",
    title="XXXXXXXX XXXXXXXXXXXX",
    linewidth=1.5,
    grid=true,
    size=(800, 600)
)

scatter!(
    plt2,
    [x_star],
    [y_star],
    markersize=8,
    label="XXXXXXXXXXXXXXXX XXXX"
)

```



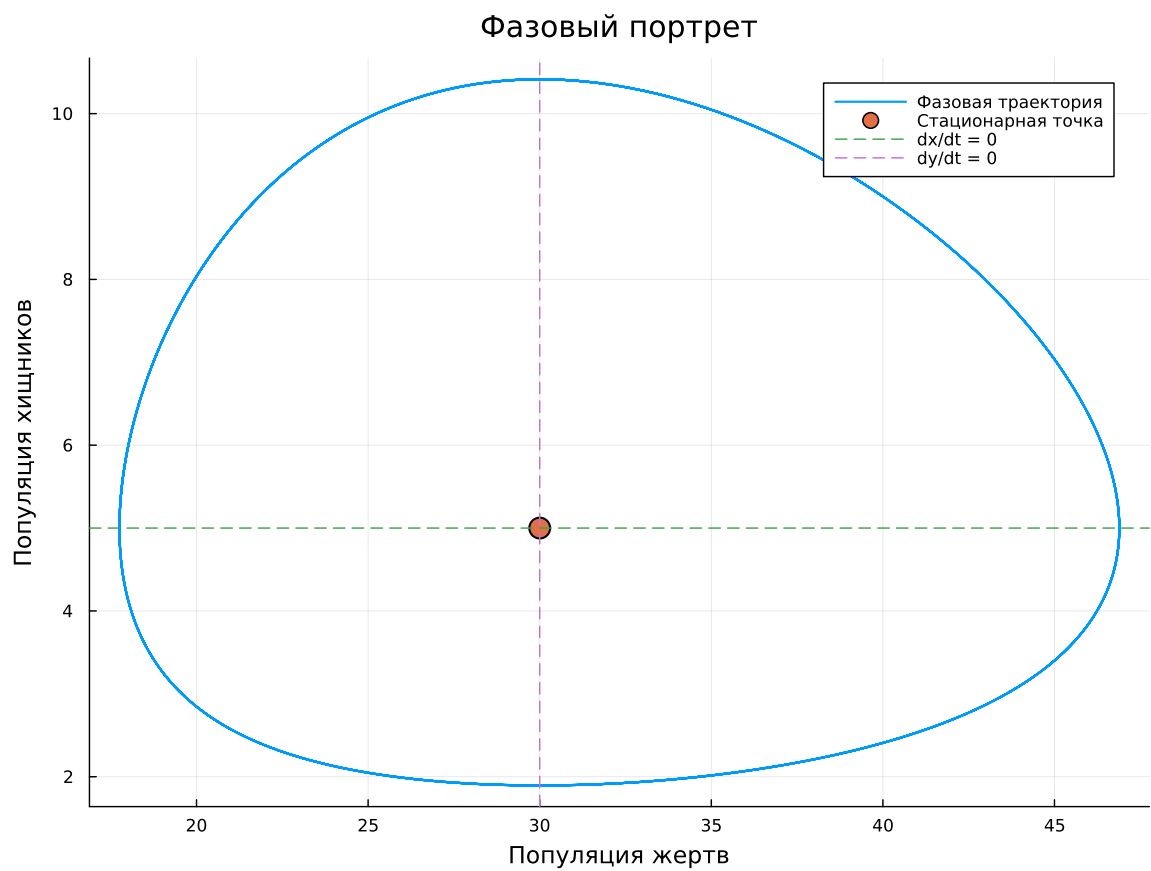
Изоклины согласно формулам модели: $dx/dt=0 \rightarrow y=\alpha/\beta$ $dy/dt=0 \rightarrow x=\gamma/\delta$

```
hline!(
    plt2,
    [y_star],
    linestyle=:dash,
    label="dx/dt = 0"
)

vline!(
    plt2,
    [x_star],
    linestyle=:dash,
```

```
label="dy/dt = 0"
```

```
)
```



3. Производные

```
plt3 = plot(  
    df_lv.t,  
    hcat(  
        df_lv.dprey_dt,  
        df_lv.dpredator_dt  
    ),  
    label=[  
        L"dx/dt"  
    ]
```

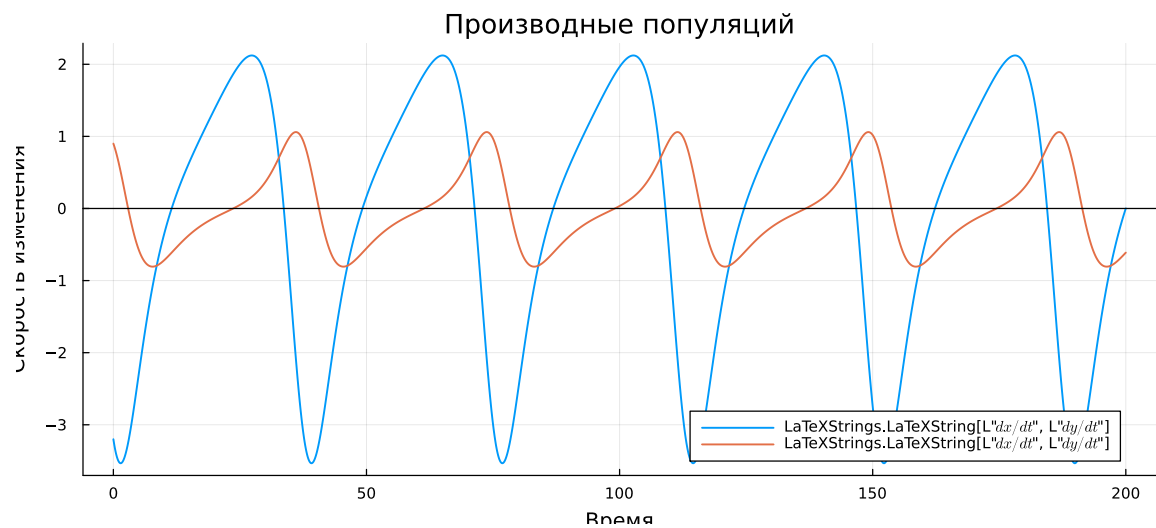


```

        L"dy/dt"
    ],
    xlabel="Время",
    ylabel="Скорость изменения",
    title="Производные популяций",
    linewidth=1.5,
    grid=true,
    size=(900, 400)
)

hline!(
    plt3,
    [0],
    color=:black,
    label=false
)

```



4. Относительные изменения

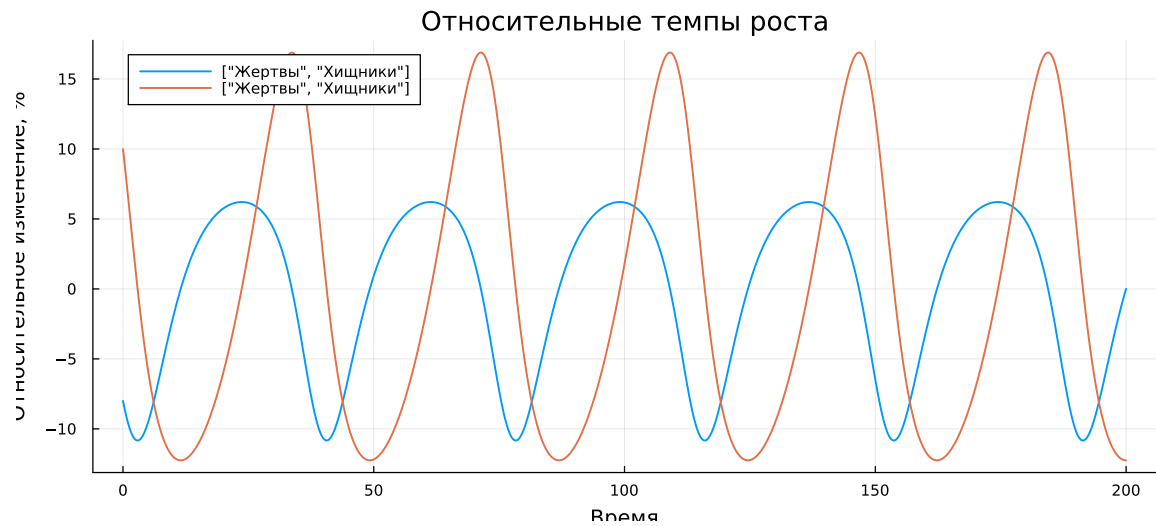
```

df_lv[!, :prey_pct_change] =
    df_lv.dprey_dt ./ df_lv.prey .* 100

df_lv[!, :predator_pct_change] =
    df_lv.dpredator_dt ./ df_lv.predator .* 100

plt4 = plot(
    df_lv.t,
    hcat(
        df_lv.prey_pct_change,
        df_lv.predator_pct_change
    ),
    label=[
        "XXXXXXXX"
        "XXXXXXXX"
    ],
    xlabel="XXXXX",
    ylabel="XXXXXXXXXXXXXXXX XXXXXXXX, %",
    title="XXXXXXXXXXXXXXXX XXXXX XXXXX",
    linewidth=1.5,
    grid=true,
    size=(900, 400)
)

```



9.6 Спектральный анализ

5. Спектральный анализ

```
function compute_fft(signal, sampling_dt)

    n = length(signal)

    spectrum =
        abs.(rfft(signal))

    freq =
        rfftfreq(
            n,
            1 / sampling_dt
        )

    return freq, spectrum
end
```

```

freq_pre,
spectrum_pre =
    compute_fft(
        df_lv.prey .-
        mean(df_lv.prey),
        saveat_lv
    )

freq_predator,
spectrum_predator =
    compute_fft(
        df_lv.predator .-
        mean(df_lv.predator),
        saveat_lv
    )

plt5 = plot(
    freq_pre[2:end],
    hcat(
        spectrum_pre[2:end],
        spectrum_predator[2:end]
    ),
    label=[
        "XXXXXXXX"
        "XXXXXXXX"
    ],
    xlabel="XXXXXXXX",
    ylabel="XXXXXXXXXX",

```

```

        title="XXXXXXXXXXXXXXXX XXXXXX",
        xscale=:log10,
        yscale=:log10,
        linewidth=1.5,
        grid=true,
        size=(800, 400)
    )

    idx_pre =
        argmax(
            spectrum_pre[2:end]
        ) + 1

    dominant_freq_pre =
        freq_pre[idx_pre]

    period_pre =
        1 / dominant_freq_pre

    println(
        "\nXXXXXXXXXXXXXXXX XXXXXX XXXXXX: ",
        round(
            dominant_freq_pre,
            digits=4
        )
    )

    println(

```

```

"Время между охотами: ",
round(
    period_prej,
    digits=2
)
)

```

Время между охотами: 0.025

Время между охотами: 40.02

6. Общая панель

```

plt6 = plot(
    layout=(3, 2),
    size=(1200, 900)
)

plot!(
    plt6[1],
    df_lv.t,
    df_lv.prej,
    label="Время",
    title="Время между охотами"
)

plot!(
    plt6[2],
    df_lv.t,

```

```

    df_lv.predator,
    label="XXXXXXXX",
    title="XXXXXXXXXX XXXXXXXX"
)

plot!(
    plt6[3],
    df_lv.prey,
    df_lv.predator,
    label=false,
    title="XXXXXXXX XXXXXXXX"
)

plot!(
    plt6[4],
    df_lv.t,
    hcat(
        df_lv.dprey_dt,
        df_lv.dpredator_dt
    ),
    label=["dx/dt" "dy/dt"],
    title="XXXXXXXXXX XXXXXXXXXX"
)

plot!(
    plt6[5],
    freq_pre[2:end],
    spectrum_pre[2:end],

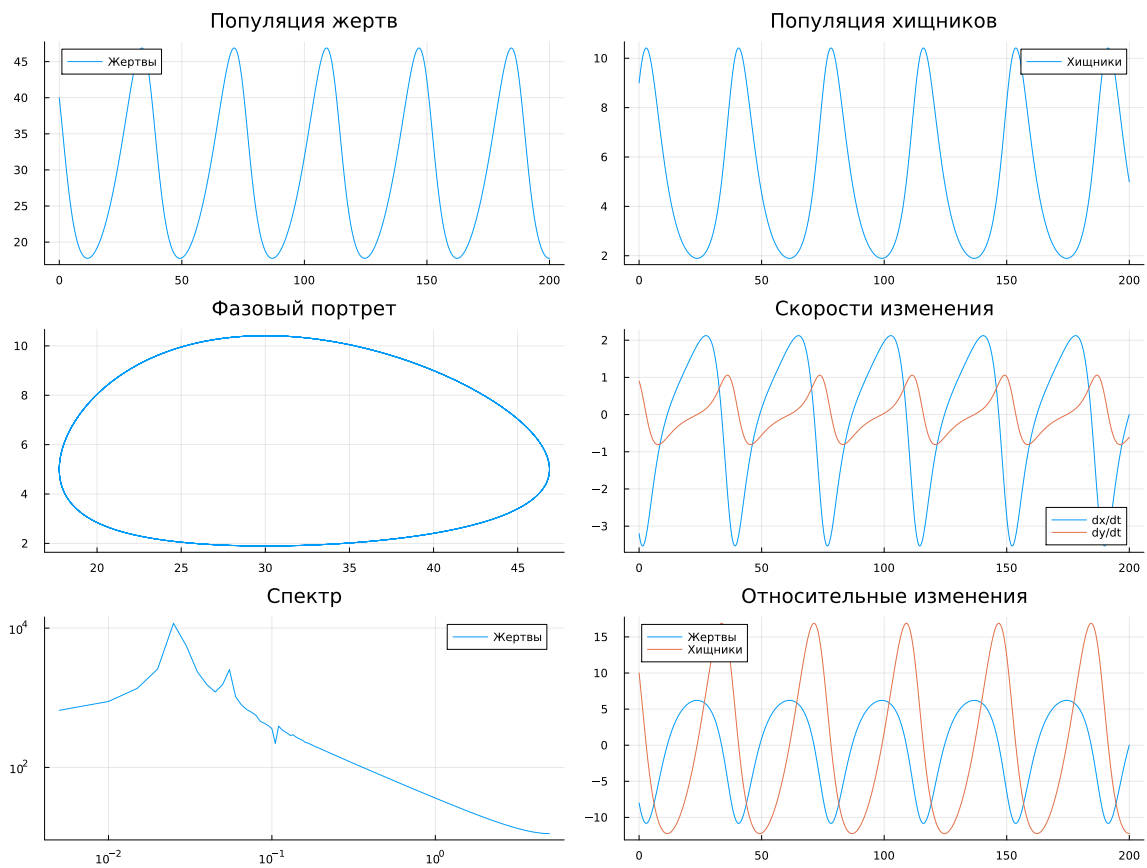
```

```

    xscale=:log10,
    yscale=:log10,
    label="XXXXXXXX",
    title="XXXXXXXX"
)

plot!(
    plt6[6],
    df_lv.t,
    hcat(
        df_lv.prey_pct_change,
        df_lv.predator_pct_change
    ),
    label=["XXXXXXXX" "XXXXXXXX"],
    title="XXXXXXXXXXXXXXXX XXXXXXXXXXXX"
)

```

9.7 Анализ результатов

Анализ результатов

```
println("\n" * "="^60)
println("XXXXXXXX XXXXXXXXXXXXXXXX")
println("="^60)

println(
    "XXXXXXXX: min=",
    round(minimum(df_lv.prey), digits=2),
    ", max=",
    round(maximum(df_lv.prey), digits=2),
```

```

    ", mean=",
    round(mean(df_lv.prey),digits=2)
)

println(
    "XXXXXXXX: min=",
    round(minimum(df_lv.predator),digits=2),
    ", max=",
    round(maximum(df_lv.predator),digits=2),
    ", mean=",
    round(mean(df_lv.predator),digits=2)
)

function find_first_peak(signal, time)

    for i in 2:length(signal)-1

        if signal[i] > signal[i-1] &&
            signal[i] > signal[i+1]

            return time[i], signal[i]
        end
    end

    return NaN, NaN
end

peak_time_prey,

```

```

peak_value_prey =
    find_first_peak(
        df_lv.prey,
        df_lv.t
    )

peak_time_predator,
peak_value_predator =
    find_first_peak(
        df_lv.predator,
        df_lv.t
    )

if !isnan(peak_time_prey) &&
    !isnan(peak_time_predator)

    phase_shift =
        peak_time_predator -
        peak_time_prey

    println(
        "\nXXXXXXXX XX XXXX: t=",
        round(peak_time_prey,digits=2),
        ", XXXXXXXX=",
        round(peak_value_prey,digits=2)
    )

    println(

```

```

        "Ловля хищника: t=",
        round(peak_time_predator,digits=2),
        ", Ловля жертвы=",
        round(peak_value_predator,digits=2)
    )

    println(
        "Сдвиг фаз: ",
        round(phase_shift,digits=2)
    )
end

```

```

=====
Ловля хищника
=====

Ловля хищника: min=17.75, max=46.89, mean=29.71
Ловля жертвы: min=1.9, max=10.41, mean=5.2

Ловля хищника Ловля жертвы: t=33.7, Ловля хищника=46.89
Ловля хищника Ловля жертвы: t=2.9, Ловля жертвы=10.41
Ловля хищника Ловля жертвы: -30.8

```

Сохранение

```

savefig(
    plt1,
    plotsdir(script_name, "lv_dynamics.png")
)

```


XXXXXXXXXXXXXXXXXXXX XXXXXXXXXXXX XXXXXXXX!

9.8 Вывод

Модель демонстрирует циклическое взаимодействие популяций и запаздывание динамики хищников относительно жертв.

10 Параметрическое исследование модели Лотки–Вольтерры

Для анализа чувствительности модели отдельно изменялись параметры естественного прироста жертв α и смертности хищников γ .

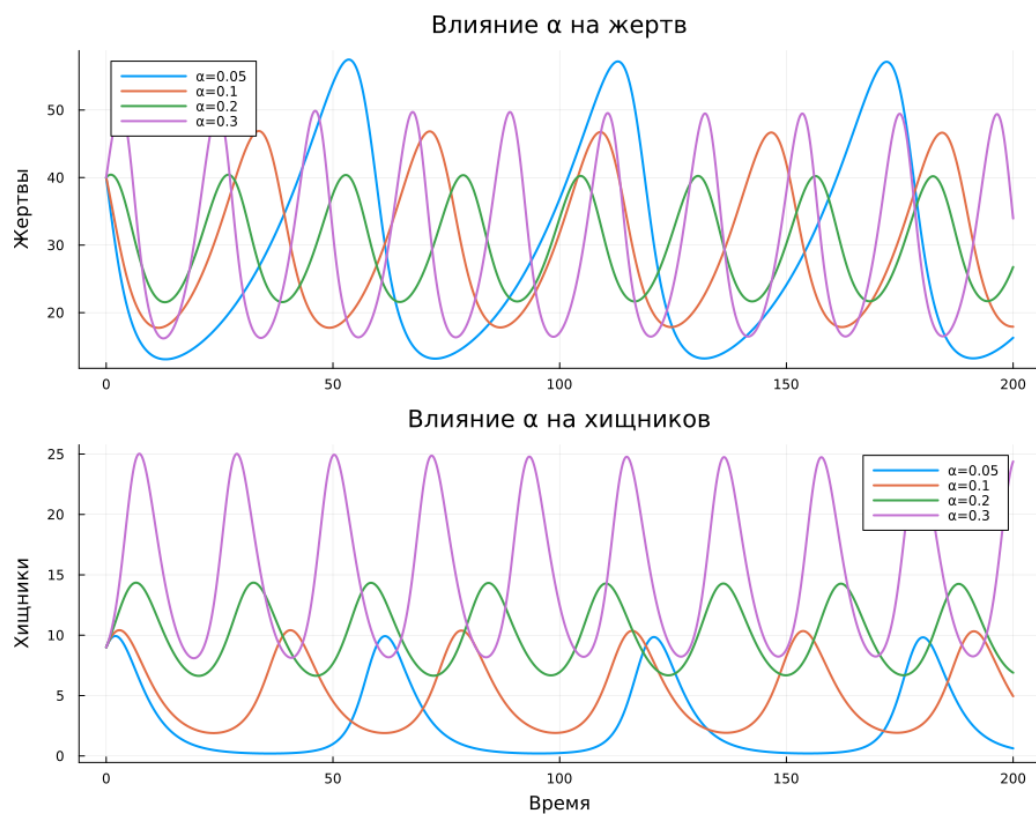
Результаты вычислений выводились в сводную таблицу.

```
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project$ julia --project=. scripts/lv_ode_params.jl
Параметрическое исследование Лотки–Вольтерры:

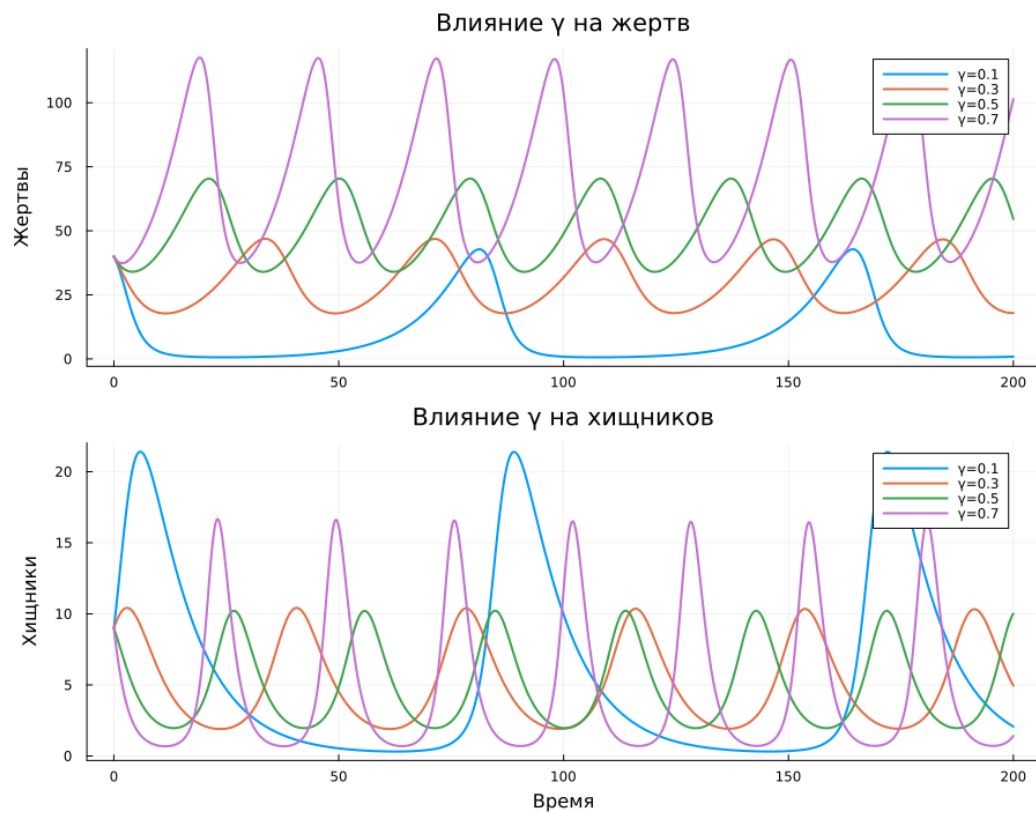
8x6 DataFrame
  Row  scan  value  prey_final  predator_final  prey_max  predator_max
  String  Float64  Float64      Float64      Float64      Float64
-----
1  alpha    0.05   16.2873      0.626478      57.469      9.93072
2  alpha    0.1    17.9235      4.94623      46.8866     10.4147
3  alpha    0.2    26.758      6.89558      40.4219     14.3519
4  alpha    0.3    33.9466     24.3865      50.045      25.0241
5  gamma    0.1     0.872333     2.06639      42.7803     21.3991
6  gamma    0.3    17.9235      4.94623      46.8866     10.4147
7  gamma    0.5    54.6107     10.0055      70.3861     10.2186
8  gamma    0.7   101.407      1.41401     117.616     16.6681

Параметрическое исследование завершено.
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project$ |
```

Влияние коэффициента α :



Дополнительно исследовалось влияние смертности хищников γ .



Параметрический код также был преобразован во все требуемые форматы.

11 Параметрическое исследование модели Лотки–Вольтерры

Автор: Нечаева Кира Андреевна

Исследуем чувствительность модели к параметрам α и γ .

```
ENV["GKSwstype"] = "100"
```

```
using DrWatson
```

```
@quickactivate "project"
```

```
using DifferentialEquations
```

```
using DataFrames
```

```
using Plots
```

```
using CSV
```

```
using JLD2
```

```
script_name =
```

```
    splitext(basename(PROGRAM_FILE))[1]
```

```
mkpath(plotsdir(script_name))
```

```
mkpath(datadir(script_name))
```

```
"/home/kiran/work/study/2026-1/2026-1==study-simulation-
```

11.1 Модель

```
function lotka_volterra!(du, u, p, t)
```

```
    x, y = u
```

```
    α, β, γ, δ = p
```

```
    du[1] = α*x - β*x*y
```

```
    du[2] = β*x*y - δ*y
```

```
    nothing
```

```
end
```

```
u0 = [40.0, 9.0]
```

```
tspan = (0.0, 200.0)
```

```
α = 0.02
```

```
β = 0.01
```

```
α_values =
```

```
    [0.05, 0.1, 0.2, 0.3]
```

```
β_values =
```

```
    [0.1, 0.3, 0.5, 0.7]
```

```
results = DataFrame(
```

```

scan=String[],
value=Float64[],
prey_final=Float64[],
predator_final=Float64[],
prey_max=Float64[],
predator_max=Float64[]
)

```

scan	value	prey_final	predator_final	prey_max	predator_max
String	Float64	Float64	Float64	Float64	Float64

11.2 Влияние α

```

plt_alpha = plot(
    layout=(2,1),
    size=(900,700)
)

for α in α_values

    p = [α, α, α, 0.3]

    prob =
        ODEProblem(
            lotka_volterra!,
            u0,
            tspan,
            p
        )

```

```

sol =
    solve(
        prob,
        Tsit5();
        saveat=0.1
    )

prey =
    [u[1] for u in sol.u]

predator =
    [u[2] for u in sol.u]

push!(
    results,
    (
        "alpha",
        ∅,
        prey[end],
        predator[end],
        maximum(prey),
        maximum(predator)
    )
)

plot!(
    plt_alpha[1],
    sol.t,

```

```

        prey,
        label="$\gamma$",
        linewidth=2,
        title="Влияние $\gamma$ на $\alpha$",
        ylabel="$\gamma$"
    )

    plot!(
        plt_alpha[2],
        sol.t,
        predator,
        label="$\gamma$",
        linewidth=2,
        title="Влияние $\gamma$ на $\alpha$ (predator)",
        xlabel="$\gamma$",
        ylabel="$\gamma$"
    )
end

```

11.3 Влияние γ

```

plt_gamma = plot(
    layout=(2,1),
    size=(900,700)
)

for  $\gamma$  in  $\gamma$ _values

```

```

p = [0.1, 0, 0, 0]

prob =
    ODEProblem(
        lotka_volterra!,
        u0,
        tspan,
        p
    )

sol =
    solve(
        prob,
        Tsit5();
        saveat=0.1
    )

prey =
    [u[1] for u in sol.u]

predator =
    [u[2] for u in sol.u]

push!(
    results,
    (
        "gamma",
        0,

```

```

        prey[end],
        predator[end],
        maximum

```
(prey),
 maximum

```
(predator)
    )
)

plot!(
    plt_gamma[1],
    sol.t,
    prey,
    label="x=$x",
    linewidth=2,
    title="Population of x over time",
    ylabel="Population of x"
)

plot!(
    plt_gamma[2],
    sol.t,
    predator,
    label="x=$x",
    linewidth=2,
    title="Population of x over time",
    xlabel="Time",
    ylabel="Population of x"
)
end

```


```

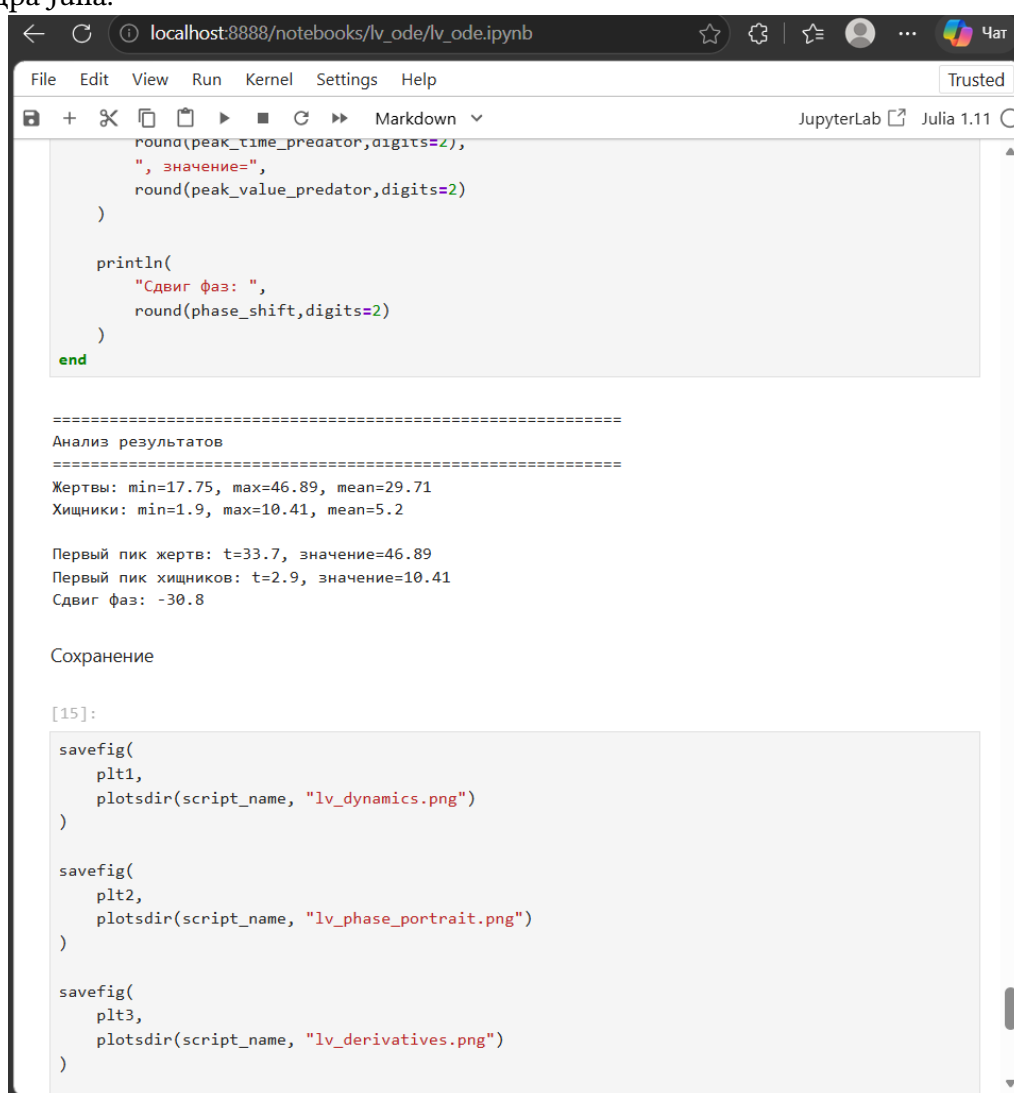

```


11.4 Результаты

```
println(  
    "XXXXXXXXXXXXXXXX XXXXXXXXXXXXXXXXXXX XXXXX-XXXXXXXXXX:\n"  
)  
  
println(results)  
  
savefig(  
    plt_alpha,  
    plotsdir(  
        script_name,  
        "lv_alpha_scan.png"  
    )  
)  
  
savefig(  
    plt_gamma,  
    plotsdir(  
        script_name,  
        "lv_gamma_scan.png"  
    )  
)  
  
CSV.write(  
    datadir(  
        script_name,  
        "lv_parameter_scan.csv"
```


12 Проверка в Jupyter Notebook

Базовый notebook модели Лотки–Вольтерры был выполнен с использованием ядра Julia.



The screenshot shows a Jupyter Notebook window titled 'localhost:8888/notebooks/lv_ode/lv_ode.ipynb'. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations and execution. The notebook is running on the Julia 1.11 kernel. The code in the cell is as follows:

```
round(peak_time_predator,digits=2),
", значение=",
round(peak_value_predator,digits=2)
)

println(
    "Сдвиг фаз: ",
    round(phase_shift,digits=2)
)
end
```

The output of the code is displayed below the code cell:

```
=====
Анализ результатов
=====
Жертвы: min=17.75, max=46.89, mean=29.71
Хищники: min=1.9, max=10.41, mean=5.2

Первый пик жертв: t=33.7, значение=46.89
Первый пик хищников: t=2.9, значение=10.41
Сдвиг фаз: -30.8

Сохранение

[15]:
savefig(
    plt1,
    plotsdir(script_name, "lv_dynamics.png")
)

savefig(
    plt2,
    plotsdir(script_name, "lv_phase_portrait.png")
)

savefig(
    plt3,
    plotsdir(script_name, "lv_derivatives.png")
)
```

Для параметрической программы также был сформирован Jupyter Notebook.

Его наличие вместе с остальными производными файлами проверено в итоговой структуре проекта.

13 Итоговая структура проекта

После выполнения всех этапов были сформированы исходные программы, чистые Julia-скрипты, документы Quarto, Jupyter Notebook, численные результаты и графики.

```
kiran@kira: ~/work/study/202
+ v
notebooks/sir_ode_params/sir_ode_params.ipynb
plots/lv_derivatives.png
plots/lv_dynamics.png
plots/lv_ode/lv_derivatives.png
plots/lv_ode/lv_dynamics.png
plots/lv_ode/lv_panel.png
plots/lv_ode/lv_phase_portrait.png
plots/lv_ode/lv_relative_changes.png
plots/lv_ode/lv_spectrum.png
plots/lv_ode_params/lv_alpha_scan.png
plots/lv_ode_params/lv_gamma_scan.png
plots/lv_panel.png
plots/lv_phase_portrait.png
plots/lv_relative_changes.png
plots/lv_spectrum.png
plots/sir_contact_scan.png
plots/sir_effective_R.png
plots/sir_infected.png
plots/sir_log_scale.png
plots/sir_main.png
plots/sir_ode/sir_effective_R.png
plots/sir_ode/sir_infected.png
plots/sir_ode/sir_log_scale.png
plots/sir_ode/sir_main.png
plots/sir_ode/sir_panel.png
plots/sir_ode/sir_percentages.png
plots/sir_ode/sir_phase_portrait.png
plots/sir_ode_params/sir_contact_scan.png
plots/sir_panel.png
plots/sir_percentages.png
plots/sir_phase_portrait.png
scripts/intro.jl
scripts/lv_ode.jl
scripts/lv_ode/lv_ode.jl
scripts/lv_ode_params.jl
scripts/sir_ode.jl
scripts/sir_ode/sir_ode.jl
scripts/sir_ode_params.jl
scripts/sir_ode_params/sir_ode_params.jl
scripts/tangle.jl
scripts/test_setup.jl
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab02/project$ |
```

Результаты лабораторной работы сохранены в системе контроля версий.

```

kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling$ git log --oneline --decorate --graph -8
* 2ff4373 (HEAD -> develop, origin/develop, github/develop) feat(lab02): add SIR and Lotka-Volterra models
* f84e30b feat(lab01): add exponential growth experiments
* 403ad16 docs(lab01): add report and presentation
* c0b4ee2 feat(lab01): add exponential growth experiments
* b426d92 Merge tag 'v1.0.0' into develop
| \
| * 9bdb09b (tag: v1.0.0, origin/master, origin/HEAD, github/master, master) Merge branch 'release/1.0.0'
| / |
| * 2b3b3e5 chore(site): add changelog
| /
* 4c77646 feat(main): make course structure
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling$ |

```

14 Заключение

В лабораторной работе были реализованы две классические системы имитационного моделирования.

Для модели SIR исследована динамика распространения инфекции и влияние среднего числа контактов на развитие эпидемии.

Для модели Лотки–Вольтерры исследованы циклические изменения популяций хищников и жертв, фазовая траектория и чувствительность системы к параметрам.

Все вычисления выполнены программно в Julia. Для обеспечения воспроизводимости использована структура DrWatson, а литературный код автоматически преобразован в Julia, Quarto и Jupyter Notebook.